

```
// =====< START >- =====
// FILE NAME: : log-tests-OLD.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests-OLD.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// =====< START >- =====

# FILE-PATH: ./src/utils/log-tests.ps1
# Make sure this .ps1 script file is explicitly saved with the
# encoding UTF-8 with BOM. If saved as raw UTF-8 without BOM,
# PS 5.1 will misinterpret string literals.
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
#=====
# MODEL: Business Logic & Data Structures
#=====

class ProjectModel {
# Only responsible for resolving paths and managing file actions
static [string] GetProjectRootPath() {
# Using local assignment to satisfy the PS 5.1 compiler context
$scriptRoot = if ($PSScriptRoot) { $PSScriptRoot } else { "." }
$currentDir = [System.IO.Path]::GetFullPath($scriptRoot)
$firstPackageJsonFound = $null
while ($currentDir -and (Test-Path $currentDir)) {
if (
(Test-Path (Join-Path $currentDir ".git")) -or
(Test-Path (Join-Path $currentDir "package.json")) -or
(Test-Path (Join-Path $currentDir "tsconfig.json")) -or
(Test-Path (Join-Path $currentDir "bun.lock"))
) {
return $currentDir
}
if ((Test-Path (Join-Path $currentDir "package.json")) -and (-not $firstPackageJsonFound)) {
$firstPackageJsonFound = $currentDir
}
$parentDir = Split-Path $currentDir -Parent
if ($parentDir -eq $currentDir) { break }
$currentDir = $parentDir
}
if ($firstPackageJsonFound) { return $firstPackageJsonFound }
return $scriptRoot
}

static [PSCustomObject] GetConfiguration() {
# Get path/directory names
$projectRootPath = [ProjectModel]::GetProjectRootPath()
$scriptPath = if ($PSScriptRoot) { $PSScriptRoot } else { "." }
# Chronological numeric sorting for files
$logTimestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$appendage = "_tests_results.log"
$logFileName = "${logTimestamp}${appendage}"
# View-centric Constants
$width = 125
$shortWidth = 50
$dash = "-" * $width
$star = "*" * $width
$shortDash = "-" * $shortWidth
$shortStar = "*" * $shortWidth
$headerStart = "=== [1/2] PROJECT ROOT TEST START ==="
$headerMid = "=== *[2/2]* WORKSPACE TEST START *=== "
$workspace = "=== *---]* WORKSPACE TEST START *=== "
$result = [PSCustomObject]@{
# Log Constants
MaxLogCount
= 5
MaxLogFileWidth = $width

```

```

MaxLogTextWidth = $shortWidth
TrackColor
= "Gray"
BoxColor
SpinnerColor
JUnitColor
= "Yellow"
= "BrightMagenta"
= "Yellow"
# Directories
ProjectRootDir = $projectRootPath
ScriptDir
= $scriptPath
# Paths
TestsPath
LogPath
LogFilePath
# File Name
Appendage
LogFileName
= Join-Path $projectRootPath "tests"
= Join-Path $projectRootPath "logs"
= Join-Path $projectRootPath "logs/$logFileName"
= $appendage
= $logFileName
# Injected Log Divider Strings
DashDivider
= "`n$shortDash`n"
StarDivider
LogHeader
LogDivider
TestDivider
}
# Ensure 'logs' directory exists
[ProjectModel]::AssertDirectory($result.LogPath)
return $result
}
static [void] AssertDirectory([string]$dir) {
if (-not (Test-Path $dir)) {
New-Item -ItemType Directory -Path $dir -Force | Out-Null
}
}
static [bool] CleanOldLogs() {
$config = [ProjectModel]::GetConfiguration()
# Prevent race conditions
if (-not (Test-Path $config.LogPath)) { return $false }
# Get files matching the pattern
$existingLogs = Get-ChildItem -Path $config.LogPath -File |
Where-Object { $_.Name -like "*${$config.Appendage}" } |
Sort-Object CreationTime # Safest way to find truly "old" logs
# Skip the newest logs, and immediately delete whatever remains
if ($existingLogs.Count -gt $config.MaxLogCount) {
$existingLogs |
Select-Object -First ($existingLogs.Count - $config.MaxLogCount) |
Remove-Item -Force
return $true
}
return $false
}
}
}
#=====
# VIEW-MODEL: State Translation & UI Definitions
#=====

```

```

$script:ViewModel = @{
    Glyph = @{
        # Core Frames
        TopLeft
        TopRight
        BotLeft
        BotRight
        = [string][char]0x256D # ⌞
        = [string][char]0x256E # ⌟
        = [string][char]0x2570 # ⌠
        = [string][char]0x256F # ⌡
        Horizontal = [string][char]0x2500 # −
        Vertical
        = [string][char]0x2502 # |
        RightTee
        LeftTee
        SqTopLeft
        SqBotLeft
        = [string][char]0x251C # ⊞
        = [string][char]0x2524 # ⊞
        = [string][char]0x250C # ⌞
        = [string][char]0x2514 # ⌠
        # Missing Clack Connectors & State Lines
        BarMiddle
        = [string][char]0x2506 # ⋮ Multi-line inputs or open streams
        BarDashed
        = [string][char]0x254C # ⋯ Divides sub-sections horizontally
        StepActive = [string][char]0x25C9 # ◎ Active radial items / focused inputs
        # Indicators & Alerts
        Check
        = [string][char]0x2713 # ✓ Success
        Cross
        Info
        Warning
        = [string][char]0x2715 # ✕ Failure / Canceled / Error state
        = [string][char]0x2139 # ⓘ Instructions / Info block tooltips
        = [string][char]0x26A0 # ⚠ Warning flags
        # Pointers & Selection Geometric shapes
        Arrowhead
        = [string][char]0x25B6 # ►
        PointerRight = [string][char]0x203A # ›
        Diamond
        = [string][char]0x25c7 # ◇
        SolidDiamond = [string][char]0x25C6 # ◆
        Circle
        = [string][char]0x25CB # ○
        SolidCircle = [string][char]0x25CF # ●
    }
    Ansi = @{
        # Core Modifiers & Controls
        Esc
        = [char]27
        Reset
        Bold
        Dim
        Underline
        Invert
        Hidden
        = "$([char]27)[0m"
        = "$([char]27)[1m"
        = "$([char]27)[2m"
        = "$([char]27)[4m"
        = "$([char]27)[7m"
    }
}

```

```
= "$([char]27)[8m"
# Standard Foreground Colors
Black
= "$([char]27)[30m"
Red
Green
Yellow
Blue
Magenta
Cyan
White
= "$([char]27)[31m"
= "$([char]27)[32m"
= "$([char]27)[33m"
= "$([char]27)[34m"
= "$([char]27)[35m"
= "$([char]27)[36m"
= "$([char]27)[37m"
# Bright Foreground Colors
BrightBlack
= "$([char]27)[90m"
Gray
BrightRed
BrightGreen
BrightYellow
BrightBlue
= "$([char]27)[90m"
= "$([char]27)[91m"
= "$([char]27)[92m"
= "$([char]27)[93m"
= "$([char]27)[94m"
BrightMagenta = "$([char]27)[95m"
BrightCyan
= "$([char]27)[96m"
BrightWhite
= "$([char]27)[97m"
# Standard Background Colors
BgBlack
= "$([char]27)[40m"
BgRed
BgGreen
BgYellow
BgBlue
BgMagenta
BgCyan
BgWhite
= "$([char]27)[41m"
= "$([char]27)[42m"
= "$([char]27)[43m"
= "$([char]27)[44m"
= "$([char]27)[45m"
= "$([char]27)[46m"
= "$([char]27)[47m"
# Bright Background Colors
BgBrightBlack = "$([char]27)[100m"
BgBrightRed
= "$([char]27)[101m"
BgBrightGreen = "$([char]27)[102m"
BgBrightYellow = "$([char]27)[103m"
BgBrightBlue
= "$([char]27)[104m"
BgBrightMagenta = "$([char]27)[105m"
BgBrightCyan
```

```

= "${[char]27}[106m"
BgBrightWhite = "${[char]27}[107m"
# Screen / Terminal Actions
ClearScreen
= "${[char]27}[2J" # Clear entire console screen window buffer
ClearLineToEnd = "${[char]27}[K" # Clear line from cursor position to right edge
CursorHome
= "${[char]27}[H" # Reset cursor back to Top-Left position (0,0)
}
}

function Initialize-AnsiTerminal {
[CmdletBinding()]
param()
# Safely enable Virtual Terminal Processing outside of a class structure
if ($PSVersionTable.PSVersion.Major -le 5) {
$Signatures = @'
[DllImport("kernel32.dll", SetLastError = true)]
public static extern IntPtr GetStdHandle(int nStdHandle);
[DllImport("kernel32.dll", SetLastError = true)]
public static extern bool GetConsoleMode(IntPtr hConsoleHandle, out uint lpMode);
[DllImport("kernel32.dll", SetLastError = true)]
public static extern bool SetConsoleMode(IntPtr hConsoleHandle, uint dwMode);
'@
$type = Add-Type -MemberDefinition $Signatures -Name "Win32Utils" -Namespace "Win32" -PassThru
2>$null
if ($type) {
$stdOutHandle = $type::GetStdHandle(-11) # STD_OUTPUT_HANDLE
$mode = 0
if ($type::GetConsoleMode($stdOutHandle, [ref]$mode)) {
$mode = $mode -bor 0x0004 # ENABLE_VIRTUAL_TERMINAL_PROCESSING
[void]$type::SetConsoleMode($stdOutHandle, $mode)
}
}
}

#=====
# VIEW: Component Rendering
#=====

function Edit-Text {
param (
[Parameter(Mandatory = $true)]
[string]$Message,
[Parameter(Mandatory = $true)]
[string]$Color
)
$resetColor = $script:ViewModel.Ansi.Reset
# If there are no internal resets, return the message unchanged
if (-not $Message.Contains($resetColor)) { return $Message }
# If a reset occurs at the very end of the string, it is intentional
# Only replace internal resets that cut off text colors mid-string
if ($Message.EndsWith($resetColor)) {
# Temporarily strip the final reset to process mid-string overrides cleanly
$subMessage = $Message.Substring(0, $Message.Length - $resetColor.Length)
if ($subMessage.Contains($resetColor)) {
$subMessage = $subMessage.Replace($resetColor, "$resetColor$Color")
}
return "$subMessage$resetColor"
}
# Replace all internal resets with the cascading fallback color
return $Message.Replace($resetColor, "$resetColor$Color")
}

function Get-Color {
param (

```

```

[Parameter(Mandatory = $true)]
$Color
)
# Safely extract string identity
$colorKey = [string]$Color
# Normalize traditional ConsoleColor names to view model keys
if ($colorKey -eq 'DarkGray') { $colorKey = 'Gray' }
if ($colorKey -eq 'Gray') { $colorKey = 'Gray' }
# Check your Ansi collection using dynamic bracket evaluation explicitly
if ($script:ViewModel.Ansi.ContainsKey($colorKey)) {
    return $script:ViewModel.Ansi[$colorKey]
}
return $script:ViewModel.Ansi.White
}

function Write-Track {
param (
[ConsoleColor]$Color = 'DarkGray',
[switch]$Intro,
[switch]$Step,
[switch]$Outro,
[switch]$Blank
)
$TopLeft = $script:ViewModel.Glyph.SqTopLeft
$BotLeft = $script:ViewModel.Glyph.SqBotLeft
$Horizontal = $script:ViewModel.Glyph.Horizontal
$Vertical = $script:ViewModel.Glyph.Vertical
$RightTee = $script:ViewModel.Glyph.RightTee
if ($Blank) {
Write-Host $Vertical -ForegroundColor $Color
return
}
$Output = @()
switch ($true) {
$Intro {
$Output += $TopLeft
break
}
$Step {
$Output += $RightTee
break
}
$Outro {
$Output += $BotLeft
break
}
}
$Output += $Horizontal
$Output += " "
if ($Output.Count -gt 0) {
$Result = $Output -join ""
Write-Host $Result -ForegroundColor $Color -NoNewline
}
}

function Write-Pill {
param (
[Parameter(Mandatory = $true)]
[string]$Text,
[ConsoleColor]$TextColor = 'Black',
[ConsoleColor]$Color = 'Magenta',
[switch]$NoNewline
)
$Left = "$([char]0xE0B6)$([char]0x2588)"
$Right = "$([char]0x2588)$([char]0xE0B4)"

```

```

Write-Host $Left -ForegroundColor $Color -NoNewline
Write-Host " $Text " -BackgroundColor $Color -ForegroundColor $TextColor -NoNewline
if ($NoNewline) {
Write-Host $Right -ForegroundColor $Color -NoNewline
}
else {
Write-Host $Right -ForegroundColor $Color
}
}

function Write-Intro {
param (
[Parameter(Mandatory = $true)]
[string]$Title,
[ConsoleColor]$TrackColor = 'DarkGray',
[switch]$NewLine
)
Write-Track -Color $TrackColor -Intro
Write-Pill -Text "$($script:ViewModel.Ansi.Bold)$Title"
if ($NewLine) {
Write-Track -Color $TrackColor -Blank
}
}

function Write-Step {
param (
[Parameter(Mandatory = $true)]
[string]$Message,
$Color = 'White',
[ConsoleColor]$TrackColor = 'DarkGray',
[switch]$NewLine
)
# Output vertical tracking line node
Write-Track -Color $TrackColor -Step
# Extract baseline ANSI color string
$fallbackColor = Get-Color -Color $Color
# Check message for internal ANSI strings
$processedMessage = Edit-Text -Message $Message -Color $fallbackColor
# Stream to terminal
Write-Host "$$fallbackColor$processedMessage$($script:ViewModel.Ansi['Reset'])"
# Continue vertical tracking line for next step
if ($NewLine) {
Write-Track -Color $TrackColor -Blank
}
}

function Write-Box {
param (
[Parameter(Mandatory = $true)]
[string]$Text,
[int]$MaxWidth = 50,
[ConsoleColor]$TrackColor = 'DarkGray',
[ConsoleColor]$BoxColor = 'Yellow'
)
$ansiRegex = "$([char]27)\[[0-9;]*[a-zA-Z]"
$cleanText = $Text -replace $ansiRegex, ""
$displayEmoji = "☐"
if ($cleanText.Length -gt ($MaxWidth - 12)) {
# Recalculate truncation length safely based on clean un-colored text
$truncLength = [math]::Max(0, ($MaxWidth - 15))
$Text = $cleanText.Substring(0, $truncLength) + "...
$cleanText = $Text
}
$innerBoxWidth = $MaxWidth - 2
$displayText = $displayEmoji + $Text
$displayCleanText = $displayEmoji + $cleanText

```

```

# Padding math calculations must use the stripped clean string length to prevent width bloat
$totalPadding = $innerBoxWidth - $displayCleanText.Length
$padLeft = [math]::Max(0, [math]::Floor($totalPadding / 2))
$padRight = [math]::Max(0, [math]::Ceiling($totalPadding / 2))
$leftSpaces = " " * $padLeft
$rightSpaces = " " * $padRight
# Line 1: Box Top Frame
Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
Write-Host ($script:ViewModel.Glyph.TopLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) +
$script:ViewModel.Glyph.TopRight) -ForegroundColor $BoxColor
# Line 2: Text Payload Line (Outputs ANSI natively without forced Foreground parameters)
Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $BoxColor -NoNewline
Write-Host "$leftSpaces$($script:ViewModel.Ansi.Magenta)$displayText$rightSpaces" -NoNewline
Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $BoxColor
# Line 3: Box Bottom Frame
Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
Write-Host ($script:ViewModel.Glyph.BotLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) +
$script:ViewModel.Glyph.BotRight) -ForegroundColor $BoxColor
Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $TrackColor
}

function Write-Outro {
param (
[Parameter(Mandatory = $true)]
[string]$Message,
[switch]$Success,
[ConsoleColor]$TrackColor = 'DarkGray'
)
# Terminate vertical track line
Write-Track -Color $TrackColor -Outro
# Extract context color
$targetColor = if ($Success) { 'Green' } else { 'Red' }
$color = Get-Color -Color $targetColor
$reset = $script:ViewModel.Ansi.Reset
# Process internal ANSI
$processedMessage = Edit-Text -Message $Message -Color $color
# Stream to terminal
Write-Host "$color$processedMessage$reset"
}

function Get-SpinnerInstance {
param (
[char[]]$Chars,
[string]$Message,
[string]$Prefix,
[string]$Color,
[string]$Reset
)
# The background worker thread does not have access to parent scopes.
# All necessary properties must be completely self-contained and passed as arguments.
return [powershell]::Create().AddScript({
param(
[char[]]$Chrs,
[string]$Msg,
[string]$Pfx,
[string]$Clr,
[string]$Rst
)
$index = 0
while ($true) {
$char = $Chrs[$index % $Chrs.Length]
[Console]::Write("`r$Pfx $Clr$char$Rst $Msg")
Start-Sleep -Milliseconds 100
$index++
}
})

```



```

}
}).AddArgument($Chars).AddArgument($Message).AddArgument($Prefix).AddArgument($Color).AddArgument($Reset)
}

function Start-Spinner {
[CmdletBinding()]
param(
[Parameter(Mandatory = $true)]
[string]$Message,
[string]$Prefix,
[string]$Color = 'BrightBlue'
)
if (-not $Prefix) {
$Prefix = "$($script:ViewModel.Glyph.BotLeft)$($script:ViewModel.Glyph.Horizontal)"
}
# Extract ANSI sequences from view model
$spinnerColor = Get-Color -Color $Color
if (-not $spinnerColor) { $spinnerColor = $script:ViewModel.Ansi.BrightBlue }
$spinnerFrames = [char[]]'⋮⋮⋮⋮'
# Build instance using our isolated creation helper function
$instance = Get-SpinnerInstance -Chars $spinnerFrames -Message $Message -Prefix $Prefix -Color
$spinnerColor -Reset $script:ViewModel.Ansi.Reset
# Trigger async invocation handle loop
[void]$instance.BeginInvoke()
return $instance
}

function Stop-Spinner {
param(
[Parameter(Mandatory = $true)]
[System.Management.Automation.PowerShell]$SpinnerInstance
)
# Force kill and dispose the asynchronous background pipeline thread safely
$SpinnerInstance.Stop()
$SpinnerInstance.Dispose()
# Reset cursor position to column zero
[Console]::Write("`r")
# Extract ANSI codes from view model map
$clearSequence = $script:ViewModel.Ansi.ClearLineToEnd
if ($clearSequence) {
[Console]::Write($clearSequence)
}
else {
$width = if ([Console]::WindowWidth -gt 0) { [Console]::WindowWidth - 1 } else { 80 }
[Console]::Write((" " * $width) + "`r")
}
}

function Get-Padding {
param (
[string]$Character = ' ',
[int]$Count = 40
)
$Character * $Count
}

#####
# CONTROLLER: Flow Orchestration
#####
$script:evaluator = {
param($match)
$removedPrefixLength = $match.Groups['prefix'].Length
$remainingPath = $match.Groups['suffix'].Value
$terminatorType = $match.Groups['type'].Value
if ($terminatorType -eq ':') {
return $remainingPath
}
}

```

```

$paddingSpaces = Get-Padding -Count $removedPrefixLength
return "${remainingPath}${paddingSpaces}"
}

function Invoke-PostProcessing {
param(
[Parameter(Mandatory = $true)]
[string]$FilePath,
[Parameter(Mandatory = $true)]
[string]$Divider
)

$maxWidth = 125
if (Test-Path $FilePath) {
$logContent = Get-Content -Path $FilePath -Raw
# 1. ALWAYS Scrub ANSI Escape sequences completely first
$ansiPattern = '\x1b\[[0-9;]*[a-zA-Z]'
$logContent = $logContent -replace $ansiPattern, ''
# 2. Scrub the stream error wrapper string and leave an intentional blank line gap
# Using `r`n ensures cross-platform line break compatibility on Windows systems
$errorWrapperPattern = '(?m)^System\.Management\.Automation\.RemoteException\r?\n?'
$logContent = $logContent -replace $errorWrapperPattern, "`r`n"
# 3. Inject divider exactly BEFORE every new workspace test block starts
$workspaceStartPattern = '(?m)^(?=[\s]*\bbun test Vertical\d)'
$logContent = $logContent -replace $workspaceStartPattern, $Divider
# 4. Strip out the monorepo workspace prefix (e.g., '@pendex/color test: ')
$workspacePrefixPattern = '(?m)^@[a-zA-Z0-9_\-]+/[a-zA-Z0-9_\-]+ test: '
$logContent = $logContent -replace $workspacePrefixPattern, ''
# 5. Replace all literal '(fail)' strings with '(Error)'
# Parentheses must be escaped with backslashes in regex
$failPattern = '\(fail\)'
$logContent = $logContent -replace $failPattern, '(Error)'
# 6. Replace all literal '(fail)' strings with '(Error)'
# Parentheses must be escaped with backslashes in regex
$failPattern = '\(pass\)'
$logContent = $logContent -replace $failPattern, "('pass')"
# 7. Shorten path and shift remaining path by the removed prefix length
$pathPattern = '(?m)^(?<prefix>.*?(?=pendex\|))(<suffix>pendex\\[^\|:~\r\n]*?)(?=[\s]*(?<type>\\|:))'
$logContent = [regex]::Replace($logContent, $pathPattern, $script:evaluator)
# 8. Format nested test descriptions by breaking onto clean, indented lines
$logContent = $logContent -replace ' > ', "`r`n`t> "
# 9. Search for two consecutive blank lines and reduce to one
# Three consecutive line breaks create two empty visual lines
$blankLinesPattern = '(?m)(\r?\n){3}'
$logContent = $logContent -replace $blankLinesPattern, "`r`n`r`n"
# Write the segmented content back to the log file
$logContent |
Out-File -FilePath $FilePath -Width $maxWidth -Encoding utf8 -Force
}

function Invoke-BunCommand {
param(
[Parameter(Mandatory = $true)]
[PSCustomObject]$Config
)

$oldPreference = $ErrorActionPreference
$ErrorActionPreference = 'Continue'
try {
# Write the initial section header directly to the log file
$Config.LogHeader | Out-File -FilePath $Config.LogFilePath -Width $Config.MaxLogFilePathWidth
-Encoding utf8
# Execute main root coverage tests
& bun test --coverage -- $Config.TestsPath 2>&1 |
ForEach-Object { $_.ToString() } |
Out-File -FilePath $Config.LogFilePath -Append -Width $Config.MaxLogFilePathWidth -Encoding utf8

```

```

$exit1 = $LASTEXITCODE
# Append the structural divider string to the log file
$Config.LogDivider | Out-File -FilePath $Config.LogFilePath -Append -Width $Config.MaxLogFilePathWidth
-Encoding utf8
# Execute monorepo workspace tests
& bun run --workspaces test 2>&1 |
ForEach-Object { $_.ToString() } |
Out-File -FilePath $Config.LogFilePath -Append -Width $Config.MaxLogFilePathWidth -Encoding utf8
$exit2 = $LASTEXITCODE
Invoke-PostProcessing -FilePath $Config.LogFilePath -Divider $Config.TestDivider
if ($exit1 -ne 0) {
return $exit1
}
return $exit2
}
catch {
throw $_
}
finally {
$ErrorActionPreference = $oldPreference
}
}

function Invoke-TestController {
param(
[Parameter(Mandatory = $true)]
[PSCustomObject]$Config
)
$greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
# Render Header
Write-Host
Write-Intro -Title "EXECUTING TESTS" -TrackColor $Config.TrackColor -NewLine
# Render Step
Write-Step -Message "Running tests and logging output to:" -Color Cyan -TrackColor $Config.TrackColor
Write-Box -Text $Config.LogFileName -MaxWidth $Config.MaxLogTextWidth -TrackColor $Config.TrackColor
-BoxColor $Config.BoxColor
# Execute Test Runner Task with Active Spinner UI
$spinner = Start-Spinner -Message "Executing bun tests..."
$exitCode = 1
try {
$exitCode = Invoke-BunCommand -Config $Config
}
finally {
Stop-Spinner -SpinnerInstance $spinner
}
# Render Footer
Write-Step -Message "$greenCheck Executing bun tests... Completed." -Color Green -TrackColor
$Config.TrackColor -NewLine
$removedLogs = [ProjectModel]::CleanOldLogs()
if ($removedLogs) {
Write-Step -Message "$greenCheck Oldest log file(s) removed to maintain retention cap." -Color
Gray -TrackColor $Config.TrackColor -NewLine
}
if ($exitCode -eq 0) {
Write-Outro -Message "🎉 Complete. All tests passed. See log for details." -Success -TrackColor
$Config.TrackColor
}
else {
Write-Outro -Message "⚠ Complete. Some tests failed. See log for details." -TrackColor
$Config.TrackColor
}
return $exitCode
}

```

```

function Remove-OldCombinedLogs {
[CmdletBinding()]
param (
[Parameter(Mandatory = $true)]
[string]$LogPath,
[Parameter(Mandatory = $true)]
[PSCustomObject]$Config,
[boolean]$IsVerbose = $false
)
$greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
# Target files matching the combined naming convention pattern
$OldCombinedFiles = Get-ChildItem -Path $LogPath -Filter "*_junit_combined_*.xml" -File
foreach ($File in $OldCombinedFiles) {
try {
Remove-Item -Path $File.FullName -Force -ErrorAction Stop
}
catch {
Write-Error "❌ Failed to remove old file $($File.FullName): $_"
}
}
if ($IsVerbose) {
Write-Step "$greenCheck Removed old combined JUnit log(s)." -Color $config.JUnitColor -TrackColor
$config.TrackColor -NewLine
}
}

function Resolve-LogDirectory {
param ([string]$Path)
if (Test-Path $Path -PathType Leaf) {
$Path = Split-Path -Path $Path -Parent
}
if (-not (Test-Path $Path -PathType Container)) {
Write-Error "❌ Error. Log directory path does not exist: $Path"
return $null
}
return $Path
}

function Invoke-XmlMerge {
param (
[System.Xml.XmlDocument]$CombinedXml,
[System.Object[]]$XmlFiles
)
foreach ($file in $XmlFiles) {
try {
$individualXml = New-Object System.Xml.XmlDocument
$individualXml.Load($file.FullName)
$testSuites = $individualXml.SelectNodes("//testsuite")
foreach ($suite in $testSuites) {
$importedSuite = $combinedXml.ImportNode($suite, $true)
$null = $combinedXml.DocumentElement.AppendChild($importedSuite)
}
}
catch {
Write-Error "❌ Error. Failed to parse XML file: $($file.FullName). $_"
}
}
}

function Merge-JunitLogs {
[CmdletBinding()]
param ()
$greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
$IsVerbose = $PSBoundParameters.ContainsKey('Verbose') -or ($VerbosePreference -eq 'Continue')

```

```

$config = [ProjectModel]::GetConfiguration()
$LogPath = Resolve-LogDirectory -Path $config.LogPath
if (-not $LogPath) { return }
if ($IsVerbose) {
Write-Host
Write-Intro -Title "MERGE JUNIT FILES" -TrackColor $config.TrackColor -NewLine
}
# Execute the requested old log cleanup BEFORE path processing and creation
Remove-OldCombinedLogs -LogPath $LogPath -Config $config -IsVerbose $IsVerbose
# Filename timestamp settings
$datestamp = Get-Date -Format "yyyyMMdd"
$timestamp = Get-Date -Format "HHmmss"
$combinedFileName = "{$datestamp}_junit_combined_{$timestamp}.xml"
$combinedFilePath = Join-Path $LogPath $combinedFileName
# Gather matching source junit test files
$xmlFiles = @(Get-ChildItem -Path $LogPath -Filter "junit*.xml" -File |
Where-Object { $_.Name -ne $combinedFileName -and $_.Extension -eq '.xml' })
if ($xmlFiles.Count -eq 0) {
if ($IsVerbose) {
Write-Host "`n Error. No JUnit XML files found to merge.`n"
}
return
}
# Initialize master XML document
$combinedXml = New-Object System.Xml.XmlDocument
$rootNode = $combinedXml.CreateElement("testsuites")
$null = $combinedXml.AppendChild($rootNode)
# Process XML tree compilation via helper
Invoke-XmlMerge -CombinedXml $combinedXml -XmlFiles $xmlFiles
# File output IO routine
try {
$combinedXml.Save($combinedFilePath)
if ($IsVerbose) {
Write-Step "Merging JUNIT test files to:" -Color $config.JUnitColor -TrackColor
$config.TrackColor
Write-Box $combinedFileName -TrackColor $config.TrackColor
}
# Purge localized source files
$xmlFiles | Remove-Item -Force
if ($IsVerbose) {
Write-Step "$greenCheck Deleted original package JUnit Logs." -Color $config.JUnitColor
-TrackColor $config.TrackColor -NewLine
}
}
catch {
Write-Error "`n Error. Failed to save the combined XML file or purge originals. $_"
}
# Final wrap-up output alerts
if ($IsVerbose) {
Write-Outro "$greenCheck Complete." -Success -TrackColor $config.TrackColor
}
else {
$fallbackColor = Get-Color -Color Green
$message = "`n$greenCheck Merging JUNIT test files complete."
$processedMessage = Edit-Text -Message $message -Color $fallbackColor
Write-Host $processedMessage
}
}
#=====
# EXECUTION ENTRY POINT
#=====
function Invoke-TerminalScriptError {
[CmdletBinding()]

```

```

param(
[Parameter(Mandatory = $true)]
[int]$Code
)
$greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
$IsVerbose = $PSBoundParameters.ContainsKey('Verbose') -or ($VerbosePreference -eq 'Continue')
if ($Code -ne 0) {
# Clear any accidental error stream pollution to keep terminal clean
$global:Error.Clear()
# Grab ANSI codes
$reset = $script:ViewModel.Ansi.Reset
$bold = $script:ViewModel.Ansi.Bold
$red = $script:ViewModel.Ansi.Red
# Check if the environment variable has a value
$scriptName = $env:npm_lifecycle_event
if (-not $scriptName) { $scriptName = "unknown" }
if ($IsVerbose) {
# Output custom error message to the terminal
Write-Host "`n${red}Error${reset}: script ${bold}`"$scriptName`${reset} exited with code
$Code"
}
}
else {
if ($IsVerbose) {
$color = Get-Color -Color Green
$message = "`n$greenCheck ALL TESTING COMPLETE."
$processedMessage = Edit-Text -Message $message -Color $color
# Output custom error message to the terminal
Write-Host $processedMessage
}
}
# Exit back to the calling process without throwing native exceptions
[Environment]::Exit($Code)
}
function Invoke-Tests {
# Establish UI terminal preferences
Initialize-AnsiTerminal
# Instantiate application configurations
$config = [ProjectModel]::GetConfiguration()
# Handover control to the MVVMC loop
$exitCode = Invoke-TestController -Config $config
# Combine junit xml files if they exist
Merge-JunitLogs
# Exit
Invoke-TerminalScriptError -Code $exitCode
}
Invoke-Tests

// ██████████ < END > - ██████████
// FILE NAME: : log-tests-OLD.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests-OLD.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < END > - ██████████

// ██████████ < START > - ██████████
// FILE NAME: : powershell-tests.ps1
// TYPE      : .PS1
// PATH      : utils/powershell-tests.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < START > - ██████████
# FILE-PATH: ./src/utils/log-tests.ps1

```

```
# Make sure this .ps1 script file is explicitly saved with the
# encoding UTF-8 with BOM. If saved as raw UTF-8 without BOM,
# PS 5.1 will misinterpret string literals.
```

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

```
#####
```

```
# VIEW-MODEL: State Translation & UI Definitions
```

```
#####
```

```
$script:ViewModel = @{
    Glyph = @{
        # Core Frames
        TopLeft      = [string][char]0x256D # ⌞
        TopRight     = [string][char]0x256E # ⌟
        BotLeft      = [string][char]0x2570 # ⌘
        BotRight     = [string][char]0x256F # ⌞
        Horizontal   = [string][char]0x2500 # ─
        Vertical     = [string][char]0x2502 # │
        RightTee     = [string][char]0x251C # ├
        LeftTee      = [string][char]0x2524 # ┤
        SqTopLeft    = [string][char]0x250C # ⌞
        SqBotLeft    = [string][char]0x2514 # ⌞

        # Missing Clack Connectors & State Lines
        BarMiddle    = [string][char]0x2506 # ┆ Multi-line inputs or open streams
        BarDashed    = [string][char]0x254C # ┆ Divides sub-sections horizontally
        StepActive   = [string][char]0x25C9 # ◎ Active radial items / focused inputs

        # Indicators & Alerts
        Check        = [string][char]0x2713 # ✓ Success
        Cross        = [string][char]0x2715 # ✕ Failure / Canceled / Error state
        Info         = [string][char]0x2139 # ⓘ Instructions / Info block tooltips
        Warning      = [string][char]0x26A0 # ⚠ Warning flags

        # Pointers & Selection Geometric shapes
        Arrowhead    = [string][char]0x25B6 # ▶
        PointerRight = [string][char]0x25BA # ›
        Diamond      = [string][char]0x25C7 # ◇
        SolidDiamond = [string][char]0x25C6 # ◆
        Circle       = [string][char]0x25CB # ○
        SolidCircle  = [string][char]0x25CF # ●
    }
}

Ansi = @{
    # Core Modifiers & Controls
    Esc      = [char]27
    Reset    = "$([char]27)[0m"
    Bold     = "$([char]27)[1m"
    Dim      = "$([char]27)[2m"
    Underline = "$([char]27)[4m"
    Invert   = "$([char]27)[7m"
    Hidden   = "$([char]27)[8m"

    # Standard Foreground Colors
    Black    = "$([char]27)[30m"
    Red      = "$([char]27)[31m"
    Green    = "$([char]27)[32m"
    Yellow   = "$([char]27)[33m"
    Blue     = "$([char]27)[34m"
    Magenta  = "$([char]27)[35m"
    Cyan     = "$([char]27)[36m"
    White    = "$([char]27)[37m"
```

```

# Bright Foreground Colors
BrightBlack   = "$([char]27)[90m"
Gray          = "$([char]27)[90m"
BrightRed     = "$([char]27)[91m"
BrightGreen   = "$([char]27)[92m"
BrightYellow  = "$([char]27)[93m"
BrightBlue    = "$([char]27)[94m"
BrightMagenta = "$([char]27)[95m"
BrightCyan    = "$([char]27)[96m"
BrightWhite   = "$([char]27)[97m"

# Standard Background Colors
BgBlack       = "$([char]27)[40m"
BgRed         = "$([char]27)[41m"
BgGreen       = "$([char]27)[42m"
BgYellow      = "$([char]27)[43m"
BgBlue        = "$([char]27)[44m"
BgMagenta     = "$([char]27)[45m"
BgCyan        = "$([char]27)[46m"
BgWhite       = "$([char]27)[47m"

# Bright Background Colors
BgBrightBlack = "$([char]27)[100m"
BgBrightRed   = "$([char]27)[101m"
BgBrightGreen = "$([char]27)[102m"
BgBrightYellow = "$([char]27)[103m"
BgBrightBlue  = "$([char]27)[104m"
BgBrightMagenta = "$([char]27)[105m"
BgBrightCyan  = "$([char]27)[106m"
BgBrightWhite = "$([char]27)[107m"

# Screen / Terminal Actions
ClearScreen    = "$([char]27)[2J" # Clear entire console screen window buffer
ClearLineToEnd = "$([char]27)[K" # Clear line from cursor position to right edge
CursorHome     = "$([char]27)[H" # Reset cursor back to Top-Left position (0,0)
}

}

# Enable ANSI/VT rendering in PS 5.1 console hosts
function Initialize-AnsiTerminal {
    [CmdletBinding()]
    param()

    # Only needed for Windows PowerShell 5.1 and earlier
    if ($PSVersionTable.PSVersion.Major -gt 5) {
        return $true
    }

    # Win32 API signatures required to toggle console VT modes
    $Signatures = @"
    [DllImport("kernel32.dll", SetLastError = true)]
    public static extern IntPtr GetStdHandle(int nStdHandle);

    [DllImport("kernel32.dll", SetLastError = true)]
    public static extern bool GetConsoleMode(IntPtr hConsoleHandle, out uint lpMode);

    [DllImport("kernel32.dll", SetLastError = true)]
    public static extern bool SetConsoleMode(IntPtr hConsoleHandle, uint dwMode);
"@

    # Load type only once —suppress warnings and errors
    $type = ("Win32.Win32Utils" -as [type])
    if (-not $type) {

```



```

        $type = Add-Type -MemberDefinition $Signatures `
            -Namespace "Win32" `
            -Name "Win32Utils" `
            -PassThru `
            -ErrorAction SilentlyContinue `
            2>$null
    }

    # If type failed to load, ANSI cannot be enabled
    if (-not $type) {
        return $false
    }

    # Standard output handle
    $STD_OUTPUT_HANDLE = -11
    $stdOut = $type::GetStdHandle($STD_OUTPUT_HANDLE)

    # Non-console hosts return 0 or -1 —avoid errors
    if ($stdOut -eq 0) {
        return $false
    }

    # Try to retrieve current console mode
    $mode = 0
    if (-not $type::GetConsoleMode($stdOut, [ref]$mode)) {
        # Non-interactive hosts will fail here
        return $false
    }

    # Enable VT processing + improved newline behavior
    $ENABLE_VTP = 0x0004 # ENABLE_VIRTUAL_TERMINAL_PROCESSING
    $newMode = $mode -bor $ENABLE_VTP

    # Apply updated mode
    $success = $type::SetConsoleMode($stdOut, $newMode)

    # Optionally ensure future consoles inherit VT support
    try {
        New-Item -Path HKCU:\Console -ErrorAction SilentlyContinue | Out-Null
        Set-ItemProperty -Path HKCU:\Console -Name VirtualTerminalLevel -Type DWord -Value 1 -ErrorAction SilentlyContinue
    }
    catch {
        # Registry access is optional; ignore any failure
    }
    return $success
}

#####
# VIEW: Component Rendering
#####
class Line {
    [string]$Prefix
    [string]$Text
    [string]$Suffix
    [bool]$NoNewLine

    Line () {
        $this.Prefix = ""
        $this.Text = ""
        $this.Suffix = ""
        $this.NoNewLine = $false
    }
}

```

```

Line (
    [string]$Text = "",
    [bool]$NoNewLine = $false
) {
    $this.Prefix = ""
    $this.Text = $Text
    $this.Suffix = ""
    $this.NoNewLine = $NoNewLine
}

Line (
    [string]$Prefix = "",
    [string]$Text = "",
    [string]$Suffix = "",
    [bool]$NoNewLine = $false
) {
    $this.Prefix = $Prefix
    $this.Text = $Text
    $this.Suffix = $Suffix
    $this.NoNewLine = $NoNewLine
}

# Automatically converts the building blocks to a unified layout string
# by override the default .ToString() method
[string]ToString() {
    # Combine the structural text segments
    [string]$assembled = "$($this.Prefix)$($this.Text)$($this.Suffix)"

    # Apply trailing carriage line termination characters natively if true
    if ($this.NoNewLine -eq $false) {
        return "$assembled`r`n"
    }
    else {
        return $assembled
    }
}

[void]Render() {
    [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
    [Console]::Write($this.ToString())
}
}

class Pill {
    hidden [Line]$Line
    hidden [hashtable]$Glyph
    hidden [hashtable]$Ansi
    hidden [ConsoleColor]$TextColor
    hidden [ConsoleColor]$PillColor

    Pill (
        [string]$Text,
        [ConsoleColor]$TextColor = [ConsoleColor]::Black,
        [ConsoleColor]$PillColor = [ConsoleColor]::Magenta,
        [bool]$NoNewline = $false
    ) {

        $this.Line = [Line]::new()
        $this.Glyph = @{
            LeftPill = [string][char]0xE0B6 # ◀
            RightPill = [string][char]0xE0B4 # ▶
            FullBlock = [string][char]0x2588 # █
        }
    }
}

```

```

    $this.Ansi = @{
        Esc    = [string][char]0x1B
        Reset  = "${[string][char]0x1B}[0m"
    }
    $this.TextColor = $TextColor
    $this.PillColor = $PillColor

    $Prefix = "${$this.Glyph.LeftPill}${$this.Glyph.FullBlock}"
    $Suffix = "${$this.Glyph.FullBlock}${$this.Glyph.RightPill}"
    $this.Init($Prefix, $Text, $Suffix, $NoNewLine)
}

# Helper method to map ConsoleColor to ANSI Foreground code
hidden [string]GetAnsiFgColor([ConsoleColor]$color) {
    $map = @{'Black' = '30'; 'DarkBlue' = '34'; 'DarkGreen' = '32'; 'DarkCyan' = '36'
            'DarkRed' = '31'; 'DarkMagenta' = '35'; 'DarkYellow' = '33'; 'Gray' = '37'
            'DarkGray' = '90'; 'Blue' = '94'; 'Green' = '92'; 'Cyan' = '96'
            'Red' = '91'; 'Magenta' = '95'; 'Yellow' = '93'; 'White' = '97'}
    return $map[$color.ToString()]
}

# Helper method to map ConsoleColor to ANSI Background code
hidden [string]GetAnsiBgColor([ConsoleColor]$color) {
    $map = @{'Black' = '40'; 'DarkBlue' = '44'; 'DarkGreen' = '42'; 'DarkCyan' = '46'
            'DarkRed' = '41'; 'DarkMagenta' = '45'; 'DarkYellow' = '43'; 'Gray' = '47'
            'DarkGray' = '100'; 'Blue' = '104'; 'Green' = '102'; 'Cyan' = '106'
            'Red' = '101'; 'Magenta' = '105'; 'Yellow' = '103'; 'White' = '107'}
    return $map[$color.ToString()]
}

[void]Init(
    [string]$Prefix,
    [string]$Text,
    [string]$Suffix,
    [bool]$NoNewLine
) {
    $ESC = $this.Ansi.Esc
    $RESET = $this.Ansi.Reset

    $textFgCode = $this.GetAnsiFgColor($this.TextColor)
    $textBgCode = $this.GetAnsiBgColor($this.PillColor)
    $pillFgCode = $this.GetAnsiFgColor($this.PillColor)

    # Build segments using ANSI colors
    $this.Line.Prefix = "$ESC[${pillFgCode}m${$Prefix}$RESET"
    $this.Line.Text = "$ESC[${textFgCode};${textBgCode}m${$Text}$RESET"
    $this.Line.Suffix = "$ESC[${pillFgCode}m${$Suffix}$RESET"
    $this.Line.NoNewLine = $NoNewLine
}

# Override the default .ToString() method
[string]ToString() {
    return $this.Line.ToString()
}

[void]Render() {
    $this.Line.Render()
}

```

```

}

class Bubble {
    hidden [hashtable]$Glyph
    hidden [string]$Text
    hidden [string]$Esc
    hidden [string]$Reset
    hidden [string]$DarkGrayFg

    Bubble ([string]$Text) {
        $this.Text = ""
        $this.Esc = [string][char]0x1B
        $this.Reset = "$($this.Esc)[0m"
        $this.DarkGrayFg = "$($this.Esc)[90m" # ANSI escape code for DarkGray Foreground

        $this.Glyph = @{
            # TopLeft      = [string][char]0x256D # ␣
            # TopRight     = [string][char]0x256E # ␣
            # BottomLeft   = [string][char]0x2570 # ␣
            # BottomRight  = [string][char]0x256F # ␣
            # Horizontal   = [string][char]0x2500 # ─
            # Vertical     = [string][char]0x2502 # │
            TopLeft      = [string][char]0x25DC # ␣
            TopRight     = [string][char]0x25DD # ␣
            BottomRight  = [string][char]0x25DE # ␣
            BottomLeft   = [string][char]0x25DF # ␣
            Horizontal   = " "
            Vertical     = " "
        }

        $this.Init($Text)
    }

    [void]Init([string]$Text) {
        $width = $Text.Length + 2
        $horizontalBar = $this.Glyph.Horizontal.ToString() * $width

        # Color strings: Borders get DarkGrayFg, Text gets default/Reset color
        $DG = $this.DarkGrayFg
        $RST = $this.Reset

        # Top border line
        $line1 = "$DG$(this.Glyph.TopLeft)$horizontalBar$(this.Glyph.TopRight)$RST`n"

        # Middle text line (Borders are dark gray, text inside is normal)
        $line2 = "$DG$(this.Glyph.Vertical)$RST $($Text) $DG$(this.Glyph.Vertical)$RST`n"

        # Bottom border line
        $line3 = "$DG$(this.Glyph.BottomLeft)$horizontalBar$(this.Glyph.BottomRight)$RST`n"

        $this.Text = "$line1$line2$line3"
    }

    # Override the default .ToString() method
    [string]ToString() {
        # Note: .ToString() usually shouldn't append newlines natively,
        # because commands like Write-Host will append their own unless -NoNewline is specified.
        return "$($this.Text)"
    }

    [void]Render() {
        [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
    }
}

```

```

        Write-Host $this.ToString()
    }
}

class Box {
    hidden [hashtable]$Glyph
    [string]$Text

    hidden [string]$TextColor
    hidden [string]$BackgroundColor
    hidden [string]$LineColor
    hidden [string]$ShadowColor
    hidden [string]$Shadow

    # Constructor
    Box([string]$text) {
        $this.Glyph = @{
            TopLeft      = [string][char]0x2554 # ┌
            TopRight     = [string][char]0x2557 # ┐
            BottomLeft   = [string][char]0x255A # └
            BottomRight  = [string][char]0x255D # ┘
            Horizontal   = [string][char]0x2550 # =
            Vertical     = [string][char]0x2551 # ||
            ShadowBlock  = [string][char]0x2588 # █
            ShadedBlock  = [string][char]0x2592 # ▒
        }

        $this.Text = $text
        $this.Shadow = $this.Glyph.ShadowBlock

        # Base initialization via 8-bit MS-DOS hardware index mappings
        $this.SetDosColors(
            '220', # Gold/Yellow Text
            '18', # IBM VGA Deep Navy Blue
            '43', # Border Cyan
            '0'  # black shadow block
        )
    }

    # Helper engine to build standard 8-bit ANSI sequences
    hidden [string]ToAnsi([string]$FgIndex, [string]$BgIndex) {
        [string]$ESC = [string][char]0x1B
        return "$ESC[38;5;${FgIndex};48;5;${BgIndex}m"
    }

    # Direct 24-bit RGB True-Color compiler to force absolute colors
    hidden [string]ToAnsiRGB([int]$R, [int]$G, [int]$B) {
        [string]$ESC = [string][char]0x1B
        return "$ESC[38;2;${R};${G};${B};49m"
    }

    # Combines 8-bit and 24-bit streams
    hidden [string]ToAnsiMixed([string]$fg, [string]$bg) {
        [string]$ESC = [string][char]0x1B
        [string]$fgPart = ""
        [string]$bgPart = ""

        # Resolve Foreground
        if ($fg -eq '0') {
            $fgPart = "38;2;0;0;0" # Pure True-Color Black FG
        }
        else {
            $fgPart = "38;5;${fg}" # Standard 8-bit FG
        }
    }
}

```

```

    }

    # Resolve Background
    if ($bg -eq '0') {
        $bgPart = "48;2;0;0;0" # Pure True-Color Black BG
    }
    else {
        $bgPart = "48;5;${bg}" # Standard 8-bit BG
    }

    return "$ESC[${fgPart};${bgPart}m"
}

hidden [string] AnsiReset() {
    [string]$ESC = [string][char]0x1B
    return "$ESC[0m"
}

# Public method maps directly into the updated string storage buckets
[void] SetDosColors([string]$fg, [string]$bg, [string]$line, [string]$shadow) {
    [string]$ESC = [string][char]0x1B

    # 1. Compile Text Color
    if ($fg -ne '0' -and $bg -ne '0') {
        $this.TextColor = $this.ToAnsi($fg, $bg)
    }
    else {
        $this.TextColor = $this.ToAnsiMixed($fg, $bg)
    }

    # 2. Compile Background Color
    if ($fg -ne '0' -and $bg -ne '0') {
        $this.BackgroundColor = $this.ToAnsi($fg, $bg)
    }
    else {
        $this.BackgroundColor = $this.ToAnsiMixed($fg, $bg)
        $this.Shadow = $this.Glyph.ShadedBlock
    }

    # 3. Compile Line Color
    if ($line -ne '0' -and $bg -ne '0') {
        $this.LineColor = $this.ToAnsi($line, $bg)
    }
    else {
        $this.LineColor = $this.ToAnsiMixed($line, $bg)
    }

    # 4. Compile Shadow Color (Utilizes your ToAnsiRGB helper directly if '0')
    if ($shadow -eq '0') {
        $this.ShadowColor = $this.ToAnsiRGB(0, 0, 0)
    }
    else {
        $this.ShadowColor = "$ESC[38;5;${shadow};49m"
    }
}

# Custom ToString override leveraging an array of Line objects
[string] ToString() {
    $lineLength = $this.Text.Length

    $ansiBorder = $this.LineColor
    $ansiText = $this.TextColor
    $ansiShadow = $this.ShadowColor

```

```

    $reset = $this.AnsiReset()

    # Build raw frame string assets
    $hLines = $this.Glyph.Horizontal * ($lineLength + 2)
    $topRow = "$($this.Glyph.TopLeft)$hLines$($this.Glyph.TopRight)"
    $bottomRow = "$($this.Glyph.BottomLeft)$hLines$($this.Glyph.BottomRight)"
    $dropShadow = $this.Shadow * ($topRow.Length - 1)

    # Instantiate lines into a strict type-safe array container
    [Line[]]$BoxLines = @(
        [Line]::new($ansiBorder, "$topRow$ansiShadow ", $reset, $false),
        [Line]::new(
            $ansiBorder,
            "$($this.Glyph.Vertical)$ansiText $($this.Text) $ansiBorder$($this.Glyph.Vertical)$ansiShadow$($this.Shadow)",
            $reset,
            $false
        ),
        [Line]::new($ansiBorder, "$bottomRow$ansiShadow$($this.Shadow)", $reset, $false),
        [Line]::new($ansiShadow, " $dropShadow", $reset, $true)
    )

    $outputBuilder = [System.Text.StringBuilder]::new()
    foreach ($line in $BoxLines) {
        [void]$outputBuilder.Append($line.ToString())
    }

    return $outputBuilder.ToString()
}

[void] Render() {
    [Console]::Write($this.ToString())
    [Console]::Write("$($this.AnsiReset())`r`n")
}
}

class Menu {
    # Define hidden properties
    hidden [hashtable]$Glyph
    hidden [string]$TextColor
    hidden [string]$BackgroundColor
    hidden [string]$LineColor
    hidden [string]$ShadowColor
    hidden [int]$Selected
    hidden [string]$Shadow

    # Public class properties
    [string]$Title
    [string[]]$Choices

    # Default Constructor
    Menu([string]$title, [string[]]$choices) {
        $this.Glyph = @{
            TopLeft      = [string][char]0x2554 # ┌
            TopRight     = [string][char]0x2557 # ┐
            BottomLeft   = [string][char]0x255A # └
            BottomRight  = [string][char]0x255D # ┘
            Horizontal    = [string][char]0x2550 # =
            Vertical      = [string][char]0x2551 # ||
            ShadowBlock   = [string][char]0x2588 # █ (Solid Pure Shadow)
            ShadedBlock  = [string][char]0x2592 # ░
        }
        $this.Title = $title
        $this.Choices = $choices
    }
}

```

```

    $this.Shadow = $this.Glyph.ShadowBlock
    $this.Selected = 0

    # Initialize default Norton Commander colors (Bright White text, Navy Blue background, Cyan lines, Black shadow)
    $this.SetDosColors('15', '18', '14', '0')
}

# Helper engine to build standard 8-bit ANSI sequences
hidden [string]ToAnsi([string]$FgIndex, [string]$BgIndex) {
    [string]$ESC = [string][char]0x1B
    return "$ESC[38;5;${FgIndex};48;5;${BgIndex}m"
}

# Direct 24-bit RGB True-Color compiler to force absolute colors
hidden [string]ToAnsiRGB([int]$R, [int]$G, [int]$B) {
    [string]$ESC = [string][char]0x1B
    return "$ESC[38;2;${R};${G};${B};49m"
}

# Combines 8-bit and 24-bit streams
hidden [string]ToAnsiMixed([string]$fg, [string]$bg) {
    [string]$ESC = [string][char]0x1B
    [string]$fgPart = ""
    [string]$bgPart = ""

    # Resolve Foreground
    if ($fg -eq '0') {
        $fgPart = "38;2;0;0;0" # Pure True-Color Black FG
    }
    else {
        $fgPart = "38;5;${fg}" # Standard 8-bit FG
    }

    # Resolve Background
    if ($bg -eq '0') {
        $bgPart = "48;2;0;0;0" # Pure True-Color Black BG
    }
    else {
        $bgPart = "48;5;${bg}" # Standard 8-bit BG
    }

    return "$ESC[${fgPart};${bgPart}m"
}

hidden [string]AnsiReset() {
    [string]$ESC = [string][char]0x1B
    return "$ESC[0m"
}

[void] SetDosColors([string]$fg, [string]$bg, [string]$line, [string]$shadow) {
    [string]$ESC = [string][char]0x1B

    # 1. Compile Text Color
    if ($fg -ne '0' -and $bg -ne '0') {
        $this.TextColor = $this.ToAnsi($fg, $bg)
    }
    else {
        $this.TextColor = $this.ToAnsiMixed($fg, $bg)
    }

    # 2. Compile Background Color
    if ($fg -ne '0' -and $bg -ne '0') {
        $this.BackgroundColor = $this.ToAnsi($fg, $bg)
    }
}

```



```

    }
    else {
        $this.BackgroundColor = $this.ToAnsiMixed($fg, $bg)
        $this.Shadow = $this.Glyph.ShadedBlock
    }

    # 3. Compile Line Color
    if ($line -ne '0' -and $bg -ne '0') {
        $this.LineColor = $this.ToAnsi($line, $bg)
    }
    else {
        $this.LineColor = $this.ToAnsiMixed($line, $bg)
    }

    # 4. Compile Shadow Color (Utilizes your ToAnsiRGB helper directly if '0')
    if ($shadow -eq '0') {
        $this.ShadowColor = $this.ToAnsiRGB(0, 0, 0)
    }
    else {
        $this.ShadowColor = "$ESC[38;5;${shadow};49m"
    }
}

# Overridden interactive ToString engine
[string] ToString() {
    $maxWidth = $this.Title.Length + 4

    foreach ($choice in $this.Choices) {
        $choiceLengthWithPadding = $choice.Length + 4
        if ($choiceLengthWithPadding -gt $maxWidth) {
            $maxWidth = $choiceLengthWithPadding
        }
    }

    # Dynamic Color assignments mapped to class string properties
    $cBorder = $this.LineColor
    $cText = $this.TextColor

    # Keep the classic active highlighted color inverted (e.g., Black text on Golden Yellow background)
    $cSelect = $this.ToAnsi('0', '11')
    $cShadow = $this.ShadowColor
    $reset = $this.AnsiReset()

    # 1. Calculate and build the Header Border containing the Title text block
    $padLeftLength = [Math]::Max(0, [Math]::Floor(($maxWidth - $this.Title.Length - 4) / 2))
    $padRightLength = [Math]::Max(0, $maxWidth - $this.Title.Length - 4 - $padLeftLength)

    $hLeft = $this.Glyph.Horizontal * $padLeftLength
    $hRight = $this.Glyph.Horizontal * $padRightLength
    $topRow = "(${this.Glyph.TopLeft})$hLeft| ${this.Title} |$hRight${this.Glyph.TopRight}"

    $sb = [System.Text.StringBuilder]::new()
    [void]$sb.AppendLine("$cBorder$topRow$cShadow $reset")

    # 2. Build Selection Rows dynamically
    for ($i = 0; $i -lt $this.Choices.Length; $i++) {
        $itemText = " ${this.Choices[$i]}"
        $padSpacesCount = $maxWidth - $itemText.Length
        $padSpaces = " " * $padSpacesCount

        if ($i -eq $this.Selected) {
            [void]$sb.AppendLine("$cBorder${this.Glyph.Vertical})$cSelect$itemText$padSpaces$cBorder${this.Glyph.Vertical})$cShadow${this.Shadow}$reset")
        }
    }
}

```

```

        else {
            [void]$sb.AppendLine("$cBorder$( $this.Glyph.Vertical)$cText$itemText$padSpaces$cBorder$( $this.Glyph.Vertical)$cShadow$( $this.Shadow)$reset")
        }
    }

    # 3. Bottom Frame + Bottom Drop Shadow floor layout
    $bottomRow = $this.Glyph.BottomLeft + ($this.Glyph.Horizontal * $maxWidth) + $this.Glyph.BottomRight
    [void]$sb.AppendLine("$cBorder$bottomRow$cShadow$( $this.Shadow)$reset")

    $dropShadow = $this.Shadow * ($topRow.Length - 1)
    [void]$sb.Append("$cShadow $dropShadow$reset")

    return $sb.ToString()
}

# Loops keyboard event loops inside your active powershell session thread
[int] ShowMenu() {
    [int]$result = -1
    [int]$maxChoices = $this.Choices.Length
    [Console]::CursorVisible = $false

    $originalLeft = [Console]::CursorLeft
    $originalTop = [Console]::CursorTop

    [bool]$needsRender = $true

    while ($true) {
        if ($needsRender) {
            [Console]::SetCursorPosition($originalLeft, $originalTop)
            [Console]::Write($this.ToString())
            $needsRender = $false
        }

        $pressed = [Console]::ReadKey($true)
        if ($pressed.Key -eq [ConsoleKey]::UpArrow) {
            $this.Selected = ($this.Selected - 1 + $maxChoices) % $maxChoices
            $needsRender = $true
        }
        elseif ($pressed.Key -eq [ConsoleKey]::DownArrow) {
            $this.Selected = ($this.Selected + 1) % $maxChoices
            $needsRender = $true
        }
        elseif ($pressed.Key -eq [ConsoleKey]::Enter) {
            [Console]::CursorVisible = $true
            [Console]::WriteLine("`r`n")
            $result = ($this.Selected + 1)
            break
        }
    }
    return $result
}

enum PromptType {
    Intro
    Step
    Outro
    Blank
}

class Prompt {
    hidden [hashtable]$Glyph
    hidden [hashtable]$Ansi

```

```

[string]$Message

# Default constructor (no parameters)
Prompt() {
    [Console]::OutputEncoding = [System.Text.Encoding]::UTF8

    $this.Message = ""

    $this.Glyph = @{
        # Core Frames
        TopLeft      = [string][char]0x256D # ⏏
        TopRight     = [string][char]0x256E # ⏐
        BotLeft      = [string][char]0x2570 # ⏌
        BotRight     = [string][char]0x256F # ⏎
        Horizontal   = [string][char]0x2500 # —
        Vertical     = [string][char]0x2502 # |
        RightTee     = [string][char]0x251C # ⊣
        LeftTee      = [string][char]0x2524 # ⊢
        SqTopLeft    = [string][char]0x250C # ⌞
        SqBotLeft    = [string][char]0x2514 # ⌟

        # Pill Shapes
        LeftPill      = [string][char]0xE0B6 # 0x25D6 # ◐
        RightPill     = [string][char]0xE0B4 # 0x25D7 # ◑
        FullBlock     = [string][char]0x2588 # █

        # Connectors & State Lines
        BarMiddle     = [string][char]0x2586 # ⋮ Multi-line inputs or open streams
        BarDashed     = [string][char]0x254C # ⋯ Divides sub-sections horizontally
        StepActive    = [string][char]0x25C9 # ☉ Active radial items / focused inputs

        # Indicators & Alerts
        Check         = [string][char]0x2713 # ✓ Success
        Cross         = [string][char]0x2715 # ✕ Failure / Canceled / Error state
        Info         = [string][char]0x2139 # ⓘ Instructions / Info block tooltips
        Warning       = [string][char]0x26A0 # ⚠ Warning flags

        # Pointers & Selection Geometric shapes
        Arrowhead     = [string][char]0x25B6 # ▶
        PointerRight  = [string][char]0x203A # ›
        Diamond       = [string][char]0x25c7 # ◇
        SolidDiamond  = [string][char]0x25C6 # ◆
        Circle        = [string][char]0x25CB # ○
        SolidCircle   = [string][char]0x25CF # ●

        # Additional Unicode Glyphs
        UpperHalfBlock = [string][char]0x2580 # ▀
        LowerOneEighthBlock = [string][char]0x2581 # ▁
        LowerOneQuarterBlock = [string][char]0x2582 # ▂
        LowerThreeEighthsBlock = [string][char]0x2583 # ▃
        LowerHalfBlock2 = [string][char]0x2584 # ▄
        LowerFiveEighthsBlock = [string][char]0x2585 # ▅
        LowerThreeQuartersBlock = [string][char]0x2586 # ▆
        LowerSevenEighthsBlock = [string][char]0x2587 # ▇
        LeftSevenEighthsBlock = [string][char]0x2589 # ▉
        LeftThreeQuartersBlock = [string][char]0x258A # ▊
        LeftFiveEighthsBlock = [string][char]0x258B # ▋
        LeftHalfBlock = [string][char]0x258C # ▌
        LeftThreeEighthsBlock = [string][char]0x258D # ▍
        LeftOneQuarterBlock = [string][char]0x258E # ▎
        LeftOneEighthBlock = [string][char]0x258F # ▏
        RightHalfBlock = [string][char]0x2590 # ▐
    }

```

```

LightShade      = [string][char]0x2591 # ▒
MediumShade     = [string][char]0x2592 # ▒
DarkShade       = [string][char]0x2593 # ▒
UpperOneEighthBlock = [string][char]0x2594 # ▒
RightOneEighthBlock = [string][char]0x2595 # ▒
QuadrantLowerLeft  = [string][char]0x2596 # ▒
QuadrantLowerRight = [string][char]0x2597 # ▒
QuadrantUpperLeft  = [string][char]0x2598 # ▒
QuadrantUL_LL_LR   = [string][char]0x2599 # ▒
QuadrantUL_LR      = [string][char]0x259A # ▒
QuadrantUL_UR_LL   = [string][char]0x259B # ▒
QuadrantUL_UR_LR   = [string][char]0x259C # ▒
QuadrantUpperRight = [string][char]0x259D # ▒
QuadrantUR_LL      = [string][char]0x259E # ▒
QuadrantUR_LL_LR   = [string][char]0x259F # ▒
}

```

```

$this.Ansi = @{
# Core Modifiers & Controls
Esc      = [string][char]0x1B
Reset    = "$([char]27)[0m"
Bold     = "$([char]27)[1m"
Dim      = "$([char]27)[2m"
Underline = "$([char]27)[4m"
Invert   = "$([char]27)[7m"
Hidden   = "$([char]27)[8m"

```

Standard Foreground Colors

```

Black    = "$([char]27)[30m"
Red      = "$([char]27)[31m"
Green    = "$([char]27)[32m"
Yellow   = "$([char]27)[33m"
Blue     = "$([char]27)[34m"
Magenta  = "$([char]27)[35m"
Cyan     = "$([char]27)[36m"
White    = "$([char]27)[37m"

```

Bright Foreground Colors

```

BrightBlack = "$([char]27)[90m"
Gray        = "$([char]27)[90m"
BrightRed   = "$([char]27)[91m"
BrightGreen = "$([char]27)[92m"
BrightYellow = "$([char]27)[93m"
BrightBlue  = "$([char]27)[94m"
BrightMagenta = "$([char]27)[95m"
BrightCyan  = "$([char]27)[96m"
BrightWhite = "$([char]27)[97m"

```

Standard Background Colors

```

BgBlack    = "$([char]27)[40m"
BgRed      = "$([char]27)[41m"
BgGreen    = "$([char]27)[42m"
BgYellow   = "$([char]27)[43m"
BgBlue     = "$([char]27)[44m"
BgMagenta  = "$([char]27)[45m"
BgCyan     = "$([char]27)[46m"
BgWhite    = "$([char]27)[47m"

```

Bright Background Colors

```

BgBrightBlack = "$([char]27)[100m"
BgBrightRed   = "$([char]27)[101m"
BgBrightGreen = "$([char]27)[102m"
BgBrightYellow = "$([char]27)[103m"

```

```

        BgBrightBlue   = "$([char]27)[104m"
        BgBrightMagenta = "$([char]27)[105m"
        BgBrightCyan   = "$([char]27)[106m"
        BgBrightWhite  = "$([char]27)[107m"

        # Screen / Terminal Actions
        ClearScreen     = "$([char]27)[2J" # Clear entire console screen window buffer
        ClearLineToEnd  = "$([char]27)[K"  # Clear line from cursor position to right edge
        CursorHome      = "$([char]27)[H"  # Reset cursor back to Top-Left position (0,0)
    }
}

# Helper method to map ConsoleColor to ANSI Foreground code
hidden [string]GetAnsiFgColor ([ConsoleColor]$color) {
    $map = @{'Black' = '30'; 'DarkBlue' = '34'; 'DarkGreen' = '32'; 'DarkCyan' = '36'
            'DarkRed' = '31'; 'DarkMagenta' = '35'; 'DarkYellow' = '33'; 'Gray' = '37'
            'DarkGray' = '90'; 'Blue' = '94'; 'Green' = '92'; 'Cyan' = '96'
            'Red' = '91'; 'Magenta' = '95'; 'Yellow' = '93'; 'White' = '97'}

    return $map[$color.ToString()]
}

# Helper method to map ConsoleColor to ANSI Background code
hidden [string]GetAnsiBgColor ([ConsoleColor]$color) {
    $map = @{'Black' = '40'; 'DarkBlue' = '44'; 'DarkGreen' = '42'; 'DarkCyan' = '46'
            'DarkRed' = '41'; 'DarkMagenta' = '45'; 'DarkYellow' = '43'; 'Gray' = '47'
            'DarkGray' = '100'; 'Blue' = '104'; 'Green' = '102'; 'Cyan' = '106'
            'Red' = '101'; 'Magenta' = '105'; 'Yellow' = '103'; 'White' = '107'}

    return $map[$color.ToString()]
}

# Helper engine to build ANSI sequences
hidden [string]ToAnsi([ConsoleColor]$Fg) {
    return $this.ToAnsi($Fg, [ConsoleColor]::Black)
}

hidden [string]ToAnsi([ConsoleColor]$Fg, [ConsoleColor]$Bg) {
    [string]$ESC = [string][char]0x1B
    [hashtable]$fgMap = @{'Black' = '30'; 'DarkBlue' = '34'; 'DarkGreen' = '32'; 'DarkCyan' = '36'; 'DarkRed' = '31'; 'DarkMagenta' = '35';
        'DarkYellow' = '33'; 'Gray' = '37'; 'DarkGray' = '90'; 'Blue' = '94'; 'Green' = '92'; 'Cyan' = '96'; 'Red' = '91'; 'Magenta' = '95';
        'Yellow' = '93'; 'White' = '97'}

    [hashtable]$bgMap = @{'Black' = '40'; 'DarkBlue' = '44'; 'DarkGreen' = '42'; 'DarkCyan' = '46'; 'DarkRed' = '41'; 'DarkMagenta' = '45';
        'DarkYellow' = '43'; 'Gray' = '47'; 'DarkGray' = '100'; 'Blue' = '104'; 'Green' = '102'; 'Cyan' = '106'; 'Red' = '101'; 'Magenta' = '105';
        'Yellow' = '103'; 'White' = '107'}

    # [char]0x1B creates the absolute literal ASCII Escape character required by Windows
    return "$ESC[$($fgMap[$Fg.ToString()]); $($bgMap[$Bg.ToString()])m"
}

hidden [string]AnsiReset() {
    [string]$ESC = [string][char]0x1B
    return "$ESC[0m"
}

[void]RenderBlankTrackLine() {
    [string]$result = $this.Track([PromptType]::Blank, [ConsoleColor]::DarkGray)
    [Console]::Write($result)
}

```

```

[void]RenderBlankTrackLine([ConsoleColor]$TrackColor) {
    [string]$result = $this.Track([PromptType]::Blank, $TrackColor)
    [Console]::Write($result)
}

# Overload 1: No parameters = blank line with vertical track
[string]Track () {
    return $this.Track([PromptType]::Blank)
}

# Overload 2: Prompt Type (Default: Track color DarkGray)
[string]Track ([PromptType]$Type) {
    return $this.Track($Type, [ConsoleColor]::DarkGray)
}

# Track Core Method
[string]Track ([PromptType]$Type, [ConsoleColor]$TrackColor) {
    # Line builder
    [Line]$LINE = [Line]::new()

    # Blank Track Line
    if ($Type -eq [PromptType]::Blank) {
        $LINE.Prefix = $this.ToAnsi($TrackColor)
        $LINE.Text = $this.Glyph.Vertical
        $LINE.Suffix = $this.AnsiReset()
        $LINE.NoNewLine = $true
        return $LINE
    }

    $LINE.Prefix = $this.ToAnsi($TrackColor)

    if ($Type -eq [PromptType]::Intro) {
        $LINE.Text = "($this.Glyph.SqTopLeft)$($this.Glyph.Horizontal) "
    }
    elseif ($Type -eq [PromptType]::Step) {
        $LINE.Text = "($this.Glyph.RightTee)$($this.Glyph.Horizontal) "
    }
    elseif ($Type -eq [PromptType]::Outro) {
        $LINE.Text = "($this.Glyph.SqBotLeft)$($this.Glyph.Horizontal) "
    }

    $LINE.Suffix = $this.AnsiReset()
    $LINE.NoNewLine = $true

    return $LINE.ToString()
}

# Overload 1: Minimal input (Defaults: Black text, Magenta pill, new line)
[string]Pill ([string]$Text) {
    return $this.Pill($Text, [ConsoleColor]::Black)
}

# Overload 2: Custom Text Color (Defaults: Magenta pill, new line)
[string]Pill ([string]$Text, [ConsoleColor]$TextColor) {
    return $this.Pill($Text, $TextColor, [ConsoleColor]::Magenta)
}

# Overload 3: Custom Colors (Default: new line)
[string]Pill ([string]$Text, [ConsoleColor]$TextColor, [ConsoleColor]$PillColor) {
    return $this.Pill($Text, $TextColor, $PillColor, $false)
}

# Overload 4: The Core Method (Accepts all parameters, including the switch)

```

```

[string]Pill ([string]$Text, [ConsoleColor]$TextColor, [ConsoleColor]$PillColor, [bool]$NoNewline) {
    # Instantiates original Pill class and renders it
    [Pill]$result = [Pill]::new($Text, $TextColor, $PillColor, $NoNewline)
    return $result.ToString()
}

# Overload 1: Minimal input (Defaults: White text, Magenta pill, NO blank trailing line)
[void]Intro ([string]$Title) {
    $this.Intro($Title, [ConsoleColor]::White)
}

# Overload 2: Text and Text Color (Defaults: Magenta pill, NO blank trailing line)
[void]Intro ([string]$Title, [ConsoleColor]$TextColor) {
    $this.Intro($Title, $TextColor, [ConsoleColor]::Magenta)
}

# Overload 3: Text and Pill Color (Defaults: White text, NO blank trailing line)
# NOTE: Since Overload 2 and 3 would conflict, we use a 3-parameter call for explicit variation,
# OR you can pass the Pill Color as a string here to differentiate the signatures:
[void]Intro ([string]$Title, [string]$PillColor) {
    $this.Intro($Title, [ConsoleColor]::White, [ConsoleColor]$PillColor)
}

# Overload 4: Text and NewLine flag (Defaults: White text, Magenta pill)
[void]Intro ([string]$Title, [bool]$NewLine) {
    $this.Intro($Title, [ConsoleColor]::White, [ConsoleColor]::Magenta, $NewLine)
}

# Overload 5: Custom Colors together (Defaults: NO blank trailing line)
[void]Intro ([string]$Title, [ConsoleColor]$TextColor, [ConsoleColor]$PillColor) {
    $this.Intro($Title, $TextColor, $PillColor, $false)
}

# Overload 6: Core Method (Accepts all configurations, Track color is internal)
[void]Intro ([string]$Title, [ConsoleColor]$TextColor, [ConsoleColor]$PillColor, [bool]$NewLine) {
    [Line]$LINE = [Line]::new()

    $LINE.Prefix = $this.Track([PromptType]::Intro)

    [string]$boldTitle = "$($this.Ansi.Bold)$Title$($this.Ansi.Reset())"
    $LINE.Text = $this.Pill($boldTitle, $TextColor, $PillColor)

    if ($NewLine -eq $false) {
        $LINE.Render()
    }
    else {
        $LINE.NoNewLine = $true
        $LINE.Render()
        $this.RenderBlankTrackLine()
    }
}

# Overload 1: Minimal input (Defaults: White text, DarkGray track, no spacing newline)
[void]Step([string]$Message) {
    $this.Step($Message, [ConsoleColor]::White, [ConsoleColor]::DarkGray, $false)
}

# Overload 2: New Line (Defaults: White text, DarkGray track)
[void]Step([string]$Message, [bool]$NewLine) {
    $this.Step($Message, [ConsoleColor]::White, [ConsoleColor]::DarkGray, $NewLine)
}

# Overload 3: Custom Text Color (Defaults: DarkGray track, no spacing newline)

```

```

[void]Step([string]$Message, [string]$TextColor) {
    $this.Step($Message, $TextColor, [ConsoleColor]::DarkGray, $false)
}

# Overload 4: Custom Text Color and New Line (Defaults: DarkGray track)
[void]Step([string]$Message, [string]$TextColor, [bool]$NewLine) {
    $this.Step($Message, $TextColor, [ConsoleColor]::DarkGray, $NewLine)
}

# Overload 5: Custom Text and Track Color (Defaults: no spacing newline)
[void]Step([string]$Message, [string]$TextColor, [ConsoleColor]$TrackColor) {
    $this.Step($Message, $TextColor, $TrackColor, $false)
}

# Overload 6: Core Method (Accepts all parameters and structural switches)
[void]Step([string]$Message, [string]$TextColor, [ConsoleColor]$TrackColor, [bool]$NewLine) {
    [Line]$LINE = [Line]::new()

    # Output the left-side structural step node (| - )
    $LINE.Prefix = $this.Track([PromptType]::Step, $TrackColor)

    # Extract baseline ANSI color string using our local map
    [string]$fallbackColor = ""
    if ($this.Ansi.ContainsKey($TextColor)) {
        $fallbackColor = $this.Ansi[$TextColor]
    }
    else {
        # Fallback to standard white if the provided string color map doesn't exist
        $fallbackColor = $this.Ansi.White
    }

    # Stream the formatted message text to the terminal host
    # Utilizing $this.Ansi.Reset ensures color bleeding stops instantly at line-end
    [string]$reset = $this.AnsiReset()
    $LINE.Text = "$fallbackColor$Message$reset"

    if ($NewLine -eq $false) {
        $LINE.Render()
    }
    else {
        $LINE.NoNewLine = $true
        $LINE.Render()
        $this.RenderBlankTrackLine($TrackColor)
    }
}

# Overload 1: Minimal input (Defaults to Failure/Red, DarkGray track)
[void]Outro([string]$Message) {
    $this.Outro($Message, $false, [ConsoleColor]::DarkGray)
}

# Overload 2: Success State (Defaults to DarkGray track)
[void]Outro([string]$Message, [bool]$Success) {
    $this.Outro($Message, $Success, [ConsoleColor]::DarkGray)
}

# Overload 3: Core Method (Accepts all parameters, rejects $null)
[void]Outro([string]$Message, [bool]$Success, [ConsoleColor]$TrackColor) {
    [string]$green = $this.ToAnsi('Green')
    [string]$red = $this.ToAnsi('Red')
    [string]$reset = $this.AnsiReset()
    [Line]$LINE = [Line]::new()

```



```

# The left-side structural outro node (└ )
[string]$track = $this.Track([PromptType]::Outro, $TrackColor)

if ($Success) {
    # Use green color for successful runs
    $LINE.Prefix = "$track$green"
}
else {
    # Fallback to red color for warnings/cancellations/errors
    $LINE.Prefix = "$track$red"
}
$LINE.Text = $Message
$LINE.Suffix = "$reset`r`n"
$LINE.Render()
}
}

function Edit-Text {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Message,

        [Parameter(Mandatory = $true)]
        [string]$Color
    )

    $resetColor = $script:ViewModel.Ansi.Reset

    # If there are no internal resets, return the message unchanged
    if (-not $Message.Contains($resetColor)) { return $Message }

    # If a reset occurs at the very end of the string, it is intentional
    # Only replace internal resets that cut off text colors mid-string
    if ($Message.EndsWith($resetColor)) {
        # Temporarily strip the final reset to process mid-string overrides cleanly
        $subMessage = $Message.Substring(0, $Message.Length - $resetColor.Length)

        if ($subMessage.Contains($resetColor)) {
            $subMessage = $subMessage.Replace($resetColor, "$resetColor$Color")
        }

        return "$subMessage$resetColor"
    }

    # Replace all internal resets with the cascading fallback color
    return $Message.Replace($resetColor, "$resetColor$Color")
}

function Get-Color {
    param (
        [Parameter(Mandatory = $true)]
        $Color
    )
    # Safely extract string identity
    $colorKey = [string]$Color

    # Normalize traditional ConsoleColor names to view model keys
    if ($colorKey -eq 'DarkGray') { $colorKey = 'Gray' }
    if ($colorKey -eq 'Gray') { $colorKey = 'Gray' }

    # Check your Ansi collection using dynamic bracket evaluation explicitly
    if ($script:ViewModel.Ansi.ContainsKey($colorKey)) {
        return $script:ViewModel.Ansi[$colorKey]
    }
}

```

```

    }

    return $script:ViewModel.Ansi.White
}

function Write-Box {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Text,
        [int]$MaxWidth = 50,
        [ConsoleColor]$TrackColor = 'DarkGray',
        [ConsoleColor]$BoxColor = 'Yellow'
    )

    $ansiRegex = "$([char]27)\[[0-9;]*[a-zA-Z]"
    $cleanText = $Text -replace $ansiRegex, ""

    $displayEmoji = "🌀 "
    if ($cleanText.Length -gt ($MaxWidth - 12)) {
        # Recalculate truncation length safely based on clean un-colored text
        $truncLength = [math]::Max(0, ($MaxWidth - 15))
        $Text = $cleanText.Substring(0, $truncLength) + "..."
        $cleanText = $Text
    }

    $innerBoxWidth = $MaxWidth - 2
    $displayText = $displayEmoji + $Text
    $displayCleanText = $displayEmoji + $cleanText

    # Padding math calculations must use the stripped clean string length to prevent width bloat
    $totalPadding = $innerBoxWidth - $displayCleanText.Length
    $padLeft = [math]::Max(0, [math]::Floor($totalPadding / 2))
    $padRight = [math]::Max(0, [math]::Ceiling($totalPadding / 2))

    $leftSpaces = " " * $padLeft
    $rightSpaces = " " * $padRight

    # Line 1: Box Top Frame
    Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
    Write-Host ($script:ViewModel.Glyph.TopLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) + $script:ViewModel.Glyph.TopRight)
    -ForegroundColor $BoxColor

    # Line 2: Text Payload Line (Outputs ANSI natively without forced Foreground parameters)
    Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
    Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $BoxColor -NoNewline
    Write-Host "$leftSpaces$($script:ViewModel.Ansi.Magenta)$displayText$rightSpaces" -NoNewline
    Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $BoxColor

    # Line 3: Box Bottom Frame
    Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor $TrackColor -NoNewline
    Write-Host ($script:ViewModel.Glyph.BotLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) + $script:ViewModel.Glyph.BotRight)
    -ForegroundColor $BoxColor

    Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $TrackColor
}

function Get-SpinnerInstance {
    param (
        [char[]]$Chars,
        [string]$Message,
        [string]$Prefix,
        [string]$Color,
        [string]$Reset
    )

```

```

    )
    # The background worker thread does not have access to parent scopes.
    # All necessary properties must be completely self-contained and passed as arguments.
    return [powershell]::Create().AddScript({
        param(
            [char[]]$Chrs,
            [string]$Msg,
            [string]$Pfx,
            [string]$Clr,
            [string]$Rst
        )
        $index = 0
        while ($true) {
            $char = $Chrs[$index % $Chrs.Length]
            [Console]::Write("`r$Pfx $Clr$char$Rst $Msg")
            Start-Sleep -Milliseconds 100
            $index++
        }
    }).AddArgument($Chrs).AddArgument($Message).AddArgument($Prefix).AddArgument($Color).AddArgument($Reset)
}

function Start-Spinner {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory = $true)]
        [string]$Message,
        [string]$Prefix,
        [string]$Color = 'BrightBlue'
    )

    if (-not $Prefix) {
        $Prefix = "($($script:ViewModel.Glyph.BotLeft)$($script:ViewModel.Glyph.Horizontal))"
    }

    # Extract ANSI sequences from view model
    $spinnerColor = Get-Color -Color $Color
    if (-not $spinnerColor) { $spinnerColor = $script:ViewModel.Ansi.BrightBlue }

    $spinnerFrames = [char[]]' 🌀'

    # Build instance using our isolated creation helper function
    $instance = Get-SpinnerInstance -Chars $spinnerFrames -Message $Message -Prefix $Prefix -Color $spinnerColor -Reset
    $script:ViewModel.Ansi.Reset

    # Trigger async invocation handle loop
    [void]$instance.BeginInvoke()
    return $instance
}

function Stop-Spinner {
    param(
        [Parameter(Mandatory = $true)]
        [System.Management.Automation.PowerShell]$SpinnerInstance
    )

    # Force kill and dispose the asynchronous background pipeline thread safely
    $SpinnerInstance.Stop()
    $SpinnerInstance.Dispose()

    # Reset cursor position to column zero
    [Console]::Write("`r")

    # Extract ANSI codes from view model map

```

```

$clearSequence = $script:ViewModel.Ansi.ClearLineToEnd

if ($clearSequence) {
    [Console]::Write($clearSequence)
}
else {
    $width = if ([Console]::WindowWidth -gt 0) { [Console]::WindowWidth - 1 } else { 80 }
    [Console]::Write((" " * $width) + "`r")
}
}

function Get-Padding {
    param (
        [string]$Character = ' ',
        [int]$Count = 40
    )

    $Character * $Count
}

#####
# EXECUTION ENTRY POINT
#####

function Invoke-PromptTest {
    function Invoke-TestOne {
        $Prompt = [Prompt]::new()

        #
        # TEST #1 - Custom intro, steps, and outro
        #
        Write-Host "`nTEST #1 - Custom intro, steps, and outro`n" -ForegroundColor DarkGray

        # 1. Custom intro
        $Prompt.Intro("1. START", "White", "DarkBlue")

        # 1.A. Custom step
        $Prompt.Step("1.A. FETCHING DATA", "Black", "Yellow")

        # 1.B. Custom step
        $Prompt.Step("1.B. Connecting to remote deployment nodes...", "Gray")

        # 1.C. Custom outro
        $Prompt.Outro("1.C. FINISH", "White", "DarkGreen")
    }

    function Invoke-TestTwo {
        $Prompt = [Prompt]::new()

        #
        # TEST #2 - Two different intros
        #
        Write-Host "`nTEST #2 - Two different intros`n" -ForegroundColor DarkGray

        # Example 2.A: Standard Header block, no trailing line
        $Prompt.Intro("2.A. PROD DEPLOYMENT")

        # Example 2.B: Header block with custom track line and a trailing line spacer
        $Prompt.Intro("2.B. DATABASE MIGRATION", "DarkGray", "Cyan", $true)
    }

    function Invoke-TestThree {
        $Prompt = [Prompt]::new()
    }
}

```

```

$NewLine = $false
$Success = $true

#
# TEST #3 - Console Color Test
#
Write-Host "`nTEST #3 - Console Color Test`n" -ForegroundColor DarkGray

# Execute Intro
$block = [string][char]::ConvertFromUtf32(0x2580)
$colorBar = "$block" * 10
$Prompt.Intro("3. CONSOLE COLOR TEST", $NewLine)

# Standard Color Steps
$Prompt.Step("3.A. $colorBar Black", "Black", $NewLine)
$Prompt.Step("3.B. $colorBar Red", "Red", $NewLine)
$Prompt.Step("3.C. $colorBar Green", "Green", $NewLine)
$Prompt.Step("3.D. $colorBar Yellow", "Yellow", $NewLine)
$Prompt.Step("3.E. $colorBar Blue", "Blue", $NewLine)
$Prompt.Step("3.F. $colorBar Magenta", "Magenta", $NewLine)
$Prompt.Step("3.G. $colorBar Cyan", "Cyan", $NewLine)
$Prompt.Step("3.H. $colorBar White", "White", $NewLine)

# Bright Color Steps
$Prompt.Step("3.I. $colorBar BrightWhite", "BrightWhite", $NewLine)
$Prompt.Step("3.J. $colorBar BrightCyan", "BrightCyan", $NewLine)
$Prompt.Step("3.K. $colorBar BrightMagenta", "BrightMagenta", $NewLine)
$Prompt.Step("3.L. $colorBar BrightBlue", "BrightBlue", $NewLine)
$Prompt.Step("3.M. $colorBar BrightYellow", "BrightYellow", $NewLine)
$Prompt.Step("3.N. $colorBar BrightGreen", "BrightGreen", $NewLine)
$Prompt.Step("3.O. $colorBar BrightRed", "BrightRed", $NewLine)
$Prompt.Step("3.P. $colorBar BrightBlack", "BrightBlack/Gray", $NewLine)

# Execute Outro
$Prompt.Outro("3.Q. TEST COMPLETE", $Success)
}

function Invoke-TestFour {
    # Example 4.A: Standard Header block, no trailing line
    $Bubble1 = [Bubble]::new("4.A. PROD DEPLOYMENT")
    $Bubble1.Render()

    # Example 4.B: Header block with custom track line and a trailing line spacer
    $Bubble2 = [Bubble]::new("4.B. DATABASE MIGRATION")
    $Bubble2.Render()

    # Example 4.C
    $myBox = [Box]::new("4.C. SYSTEM ERROR: INSERT DISKETTE A")
    $myBox.Render()
    Write-Host "`n"

    # Alternative 1: Classic MS-DOS Installation Wizard (White Text, Blue Screen, Red Error Box)
    $myBox.SetDosColors('15', '124', '15', '0')
    $myBox.Render()
    Write-Host "`n"

    # Alternative 2: Retro Matrix Terminal (Vivid Lime green text on Pitch Black, Dark green borders)
    $myBox.SetDosColors('46', '0', '22', '0')
    $myBox.Render()
    Write-Host "`n"
}

function Invoke-TestFive {

```

```

#
# TEST #5
#
Write-Host "`nTEST #5`n" -ForegroundColor DarkGray

function CreateMenu {
    [CmdletBinding()]
    param()

    # Interactive menu options
    $menuTitle = "TEST 5. - NORTON SYSTEM SETUP"
    $menuChoices = @(
        "1. RUN DIAGNOSTIC SCAN",
        "2. EDIT INTERRUPT VECTORS (IRQ)",
        "3. REBOOT IN SAFE MODE",
        "4. RETURN TO DOS PROMPT"
    )

    # Create interactive menu
    return [Menu]::new($menuTitle, $menuChoices)
}

function GetUserChoice {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true, Position = 0)]
        [Menu]$Menu
    )

    # Execute interactive menu
    $choice = $Menu.ShowMenu()

    # Output user choice to terminal
    Write-Host "You selected menu index option item: " -ForegroundColor DarkGray -NoNewLine
    Write-Host "$choice" -ForegroundColor Green
    Write-Host "`n"
}

function GetMenu {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true, Position = 0)]
        [Menu]$Menu,
        [Parameter(Position = 1)]
        [string[]]$ColorCodes = @('15', '18', '14', '0')
    )
    $Menu.SetDosColors(
        $ColorCodes[0],
        $ColorCodes[1],
        $ColorCodes[2],
        $ColorCodes[3]
    )
    GetUserChoice -Menu $Menu
}

# Base initialization
[Menu]$myMenu = CreateMenu

# Example #1
GetMenu -Menu $myMenu

# Alternative Color Scheme Example #2
[string[]]$colors = @('15', '124', '15', '0')

```

```

GetMenu -Menu $myMenu -ColorCodes $colors

# Alternative Color Scheme Example #3
$colors = @('220', '18', '43', '0')
GetMenu -Menu $myMenu -ColorCodes $colors

# Alternative Color Scheme Example #4
$colors = @('46', '0', '22', '0')
GetMenu -Menu $myMenu -ColorCodes $colors
}

# Execute Tests
#Invoke-TestOne
#Invoke-TestTwo
#Invoke-TestThree
Invoke-TestFour
#Invoke-TestFive
}

function Invoke-Tests {
    # Establish UI terminal preferences
    Initialize-AnsiTerminal

    Invoke-PromptTest
}

Invoke-Tests

// =====< END >-=====
// FILE NAME: : powershell-tests.ps1
// TYPE      : .PS1
// PATH      : utils/powershell-tests.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : log-tests-v01.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests-v01.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// =====< START >-=====
# FILE-PATH: ./src/utils/log-tests.ps1

# Make sure this .ps1 script file is explicitly saved with the
# encoding UTF-8 with BOM. If saved as raw UTF-8 without BOM,
# PS 5.1 will misinterpret string literals.

[Console]::OutputEncoding = [System.Text.Encoding]::UTF8

#=====
# MODEL: Business Logic & Data Structures
#=====

class ProjectModel {
    # Only responsible for resolving paths and managing file actions
    static [string] GetProjectRootPath() {
        # Using local assignment to satisfy the PS 5.1 compiler context
        $scriptRoot = if ($PSScriptRoot) { $PSScriptRoot } else { "." }
        $currentDir = [System.IO.Path]::GetFullPath($scriptRoot)
        $firstPackageJsonFound = $null

        while ($currentDir -and (Test-Path $currentDir)) {
            if (
                (Test-Path (Join-Path $currentDir ".git")) -or

```

```

        (Test-Path (Join-Path $currentDir "package.json")) -or
        (Test-Path (Join-Path $currentDir "tsconfig.json")) -or
        (Test-Path (Join-Path $currentDir "bun.lock"))
    ) {
        return $currentDir
    }

    if ((Test-Path (Join-Path $currentDir "package.json"))
-and (-not $firstPackageJsonFound)) {
        $firstPackageJsonFound = $currentDir
    }

    $parentDir = Split-Path $currentDir -Parent
    if ($parentDir -eq $currentDir) { break }
    $currentDir = $parentDir
}

if ($firstPackageJsonFound) { return $firstPackageJsonFound }
return $scriptRoot
}

static [PSCustomObject] GetConfiguration() {
    # Get path/directory names
    $projectRootPath = [ProjectModel]::GetProjectRootPath()
    $scriptPath = if ($PSScriptRoot) { $PSScriptRoot } else { "." }

    # Chronological numeric sorting for files
    $logTimestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
    $appendage = "_tests_results.log"
    $logFileName = "${logTimestamp}${appendage}"

    # View-centric Constants
    $width = 125
    $shortWidth = 50
    $dash = "-" * $width
    $star = "*" * $width
    $shortDash = "-" * $shortWidth
    $shortStar = "*" * $shortWidth
    $headerStart = "=== [1/2] PROJECT ROOT TEST START ==="
    $headerMid = "=== *[2/2]* WORKSPACE TEST START *=== "
    $workspace = "=== *---* WORKSPACE TEST START *=== "

    $result = [PSCustomObject]@{
        # Log Constants
        MaxLogCount      = 5
        MaxLogFileWidth  = $width
        MaxLogTextWidth  = $shortWidth
        TrackColor       = "Gray"
        BoxColor         = "Yellow"
        SpinnerColor     = "BrightMagenta"
        JUnitColor       = "Yellow"

        # Directories
        ProjectRootDir   = $projectRootPath
        ScriptDir        = $scriptPath

        # Paths
        TestsPath        = Join-Path $projectRootPath "tests"
        LogPath          = Join-Path $projectRootPath "logs"
        LogFilePath      = Join-Path $projectRootPath "logs/$logFileName"

        # File Name
        Appendage        = $appendage
    }
}

```



```

        LogFileName      = $logFileName

        # Injected Log Divider Strings
        DashDivider      = "`n$shortDash`n"
        StarDivider       = "`n$shortStar`n"
        LogHeader         = "`n$dash`n$headerStart`n$dash`n"
        LogDivider        = "`n$dash`n$headerMid`n$dash`n"
        TestDivider       = "`n`n$star`n$star`n$workspace`n$star`n$star`n`n`n"
    }

    # Ensure 'logs' directory exists
    [ProjectModel]::AssertDirectory($result.LogPath)

    return $result
}

static [void] AssertDirectory([string]$dir) {
    if (-not (Test-Path $dir)) {
        New-Item -ItemType Directory -Path $dir -Force | Out-Null
    }
}

static [bool] CleanOldLogs() {
    $config = [ProjectModel]::GetConfiguration()

    # Prevent race conditions
    if (-not (Test-Path $config.LogPath)) { return $false }

    # Get files matching the pattern
    $existingLogs = Get-ChildItem -Path $config.LogPath -File |
    Where-Object { $_.Name -like "**$($config.Appendage)" } |
    Sort-Object CreationTime # Safest way to find truly "old" logs

    # Skip the newest logs, and immediately delete whatever remains
    if ($existingLogs.Count -gt $config.MaxLogCount) {
        $existingLogs |
        Select-Object -First ($existingLogs.Count - $config.MaxLogCount) |
        Remove-Item -Force
        return $true
    }
    return $false
}
}

#####
# VIEW-MODEL: State Translation & UI Definitions
#####
$script:ViewModel = @{
    Glyph = @{
        # Core Frames
        TopLeft      = [string][char]0x256D # ␣
        TopRight     = [string][char]0x256E # ␣
        BotLeft      = [string][char]0x2570 # ␣
        BotRight     = [string][char]0x256F # ␣
        Horizontal   = [string][char]0x2500 # -
        Vertical     = [string][char]0x2502 # |
        RightTee     = [string][char]0x251C # ⊣
        LeftTee      = [string][char]0x2524 # ⊢
        SqTopLeft    = [string][char]0x250C # ␣
        SqBotLeft    = [string][char]0x2514 # ␣

        # Missing Clack Connectors & State Lines
        BarMiddle    = [string][char]0x2506 # ␣ Multi-line inputs or

```

```

open streams
    BarDashed    = [string][char]0x254C # -- Divides sub-sections
horizontally
    StepActive   = [string][char]0x25C9 # ◎Active radial items /
focused inputs

    # Indicators & Alerts
    Check        = [string][char]0x2713 # ✓Success
    Cross         = [string][char]0x2715 # ✕Failure / Canceled / Error state
    Info          = [string][char]0x2139 # i Instructions / Info
block tooltips
    Warning       = [string][char]0x26A0 # ▲ Warning flags

    # Pointers & Selection Geometric shapes
    Arrowhead     = [string][char]0x25B6 # ►
    PointerRight  = [string][char]0x283A # ›
    Diamond        = [string][char]0x25c7 # ◇
    SolidDiamond   = [string][char]0x25C6 # ◆
    Circle         = [string][char]0x25CB # ○
    SolidCircle    = [string][char]0x25CF # ●
}
Ansi = @{
    # Core Modifiers & Controls
    Esc        = [char]27
    Reset      = "$([char]27)[0m"
    Bold       = "$([char]27)[1m"
    Dim        = "$([char]27)[2m"
    Underline  = "$([char]27)[4m"
    Invert     = "$([char]27)[7m"
    Hidden     = "$([char]27)[8m"

    # Standard Foreground Colors
    Black      = "$([char]27)[30m"
    Red        = "$([char]27)[31m"
    Green      = "$([char]27)[32m"
    Yellow     = "$([char]27)[33m"
    Blue       = "$([char]27)[34m"
    Magenta    = "$([char]27)[35m"
    Cyan       = "$([char]27)[36m"
    White      = "$([char]27)[37m"

    # Bright Foreground Colors
    BrightBlack = "$([char]27)[90m"
    Gray        = "$([char]27)[90m"
    BrightRed   = "$([char]27)[91m"
    BrightGreen = "$([char]27)[92m"
    BrightYellow = "$([char]27)[93m"
    BrightBlue  = "$([char]27)[94m"
    BrightMagenta = "$([char]27)[95m"
    BrightCyan  = "$([char]27)[96m"
    BrightWhite = "$([char]27)[97m"

    # Standard Background Colors
    BgBlack     = "$([char]27)[40m"
    BgRed       = "$([char]27)[41m"
    BgGreen     = "$([char]27)[42m"
    BgYellow    = "$([char]27)[43m"
    BgBlue      = "$([char]27)[44m"
    BgMagenta   = "$([char]27)[45m"
    BgCyan      = "$([char]27)[46m"
    BgWhite     = "$([char]27)[47m"

    # Bright Background Colors

```

```

        BgBrightBlack   = "$([char]27)[100m"
        BgBrightRed     = "$([char]27)[101m"
        BgBrightGreen   = "$([char]27)[102m"
        BgBrightYellow  = "$([char]27)[103m"
        BgBrightBlue    = "$([char]27)[104m"
        BgBrightMagenta = "$([char]27)[105m"
        BgBrightCyan    = "$([char]27)[106m"
        BgBrightWhite   = "$([char]27)[107m"

        # Screen / Terminal Actions
        ClearScreen      = "$([char]27)[2J" # Clear entire console
screen window buffer
        ClearLineToEnd  = "$([char]27)[K" # Clear line from cursor
position to right edge
        CursorHome      = "$([char]27)[H" # Reset cursor back to
Top-Left position (0,0)
    }
}

function Initialize-AnsiTerminal {
    [CmdletBinding()]
    param()

    # Safely enable Virtual Terminal Processing outside of a class structure
    if ($PSVersionTable.PSVersion.Major -le 5) {
        $Signatures = @"
        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern IntPtr GetStdHandle(int nStdHandle);
        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern bool GetConsoleMode(IntPtr
hConsoleHandle, out uint lpMode);
        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern bool SetConsoleMode(IntPtr
hConsoleHandle, uint dwMode);
"@
        $type = Add-Type -MemberDefinition $Signatures -Name
"Win32Utils" -Namespace "Win32" -PassThru 2>$null
        if ($type) {
            $stdOutHandle = $type::GetStdHandle(-11) # STD_OUTPUT_HANDLE
            $mode = 0
            if ($type::GetConsoleMode($stdOutHandle, [ref]$mode)) {
                $mode = $mode -bor 0x0004 # ENABLE_VIRTUAL_TERMINAL_PROCESSING
                [void]$type::SetConsoleMode($stdOutHandle, $mode)
            }
        }
    }
}

#####
# VIEW: Component Rendering
#####

function Edit-Text {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Message,

        [Parameter(Mandatory = $true)]
        [string]$Color
    )

    $resetColor = $script:ViewModel.Ansi.Reset

```

```

# If there are no internal resets, return the message unchanged
if (-not $Message.Contains($resetColor)) { return $Message }

# If a reset occurs at the very end of the string, it is intentional
# Only replace internal resets that cut off text colors mid-string
if ($Message.EndsWith($resetColor)) {
    # Temporarily strip the final reset to process mid-string
    overrides cleanly
    $subMessage = $Message.Substring(0, $Message.Length -
    $resetColor.Length)

    if ($subMessage.Contains($resetColor)) {
        $subMessage = $subMessage.Replace($resetColor, "$resetColor$Color")
    }

    return "$subMessage$resetColor"
}

# Replace all internal resets with the cascading fallback color
return $Message.Replace($resetColor, "$resetColor$Color")
}

function Get-Color {
    param (
        [Parameter(Mandatory = $true)]
        $Color
    )
    # Safely extract string identity
    $colorKey = [string]$Color

    # Normalize traditional ConsoleColor names to view model keys
    if ($colorKey -eq 'DarkGray') { $colorKey = 'Gray' }
    if ($colorKey -eq 'Gray') { $colorKey = 'Gray' }

    # Check your Ansi collection using dynamic bracket evaluation explicitly
    if ($script:ViewModel.Ansi.ContainsKey($colorKey)) {
        return $script:ViewModel.Ansi[$colorKey]
    }

    return $script:ViewModel.Ansi.White
}

function Write-Track {
    param (
        [ConsoleColor]$Color = 'DarkGray',
        [switch]$Intro,
        [switch]$Step,
        [switch]$Outro,
        [switch]$Blank
    )

    $TopLeft = $script:ViewModel.Glyph.SqTopLeft
    $BotLeft = $script:ViewModel.Glyph.SqBotLeft
    $Horizontal = $script:ViewModel.Glyph.Horizontal
    $Vertical = $script:ViewModel.Glyph.Vertical
    $RightTee = $script:ViewModel.Glyph.RightTee

    if ($Blank) {
        Write-Host $Vertical -ForegroundColor $Color
        return
    }

    $Output = @()

```

```

switch ($true) {
    $Intro {
        $Output += $TopLeft
        break
    }
    $Step {
        $Output += $RightTee
        break
    }
    $Outro {
        $Output += $BotLeft
        break
    }
}

$Output += $Horizontal
$Output += " "

if ($Output.Count -gt 0) {
    $Result = $Output -join ""
    Write-Host $Result -ForegroundColor $Color -NoNewline
}
}

function Write-Pill {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Text,
        [ConsoleColor]$TextColor = 'Black',
        [ConsoleColor]$Color = 'Magenta',
        [switch]$NoNewline
    )
    $Left = "$([char]0xE0B6)$([char]0x2588)"
    $Right = "$([char]0x2588)$([char]0xE0B4)"
    Write-Host $Left -ForegroundColor $Color -NoNewline
    Write-Host " $Text " -BackgroundColor $Color -ForegroundColor
$TextColor -NoNewline

    if ($NoNewline) {
        Write-Host $Right -ForegroundColor $Color -NoNewline
    }
    else {
        Write-Host $Right -ForegroundColor $Color
    }
}

function Write-Intro {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Title,
        [ConsoleColor]$TrackColor = 'DarkGray',
        [switch]$NewLine
    )

    Write-Track -Color $TrackColor -Intro
    Write-Pill -Text "$($script:ViewModel.Ansi.Bold)$Title"

    if ($NewLine) {
        Write-Track -Color $TrackColor -Blank
    }
}

function Write-Step {

```

```

param (
    [Parameter(Mandatory = $true)]
    [string]$Message,
    $Color = 'White',
    [ConsoleColor]$TrackColor = 'DarkGray',
    [switch]$NewLine
)
# Output vertical tracking line node
Write-Track -Color $TrackColor -Step

# Extract baseline ANSI color string
$fallbackColor = Get-Color -Color $Color

# Check message for internal ANSI strings
$processedMessage = Edit-Text -Message $Message -Color $fallbackColor

# Stream to terminal
Write-Host "$fallbackColor$processedMessage$($script:ViewModel.Ansi['Reset'])"

# Continue vertical tracking line for next step
if ($NewLine) {
    Write-Track -Color $TrackColor -Blank
}
}

function Write-Box {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Text,
        [int]$MaxWidth = 50,
        [ConsoleColor]$TrackColor = 'DarkGray',
        [ConsoleColor]$BoxColor = 'Yellow'
    )

    $ansiRegex = "$([char]27)[[0-9;]*[a-zA-Z]"
    $cleanText = $Text -replace $ansiRegex, ""

    $displayEmoji = "📦 "
    if ($cleanText.Length -gt ($MaxWidth - 12)) {
        # Recalculate truncation length safely based on clean un-colored text
        $truncLength = [math]::Max(0, ($MaxWidth - 15))
        $Text = $cleanText.Substring(0, $truncLength) + "..."
        $cleanText = $Text
    }

    $innerBoxWidth = $MaxWidth - 2
    $displayText = $displayEmoji + $Text
    $displayCleanText = $displayEmoji + $cleanText

    # Padding math calculations must use the stripped clean string
    length to prevent width bloat
    $totalPadding = $innerBoxWidth - $displayCleanText.Length
    $padLeft = [math]::Max(0, [math]::Floor($totalPadding / 2))
    $padRight = [math]::Max(0, [math]::Ceiling($totalPadding / 2))

    $leftSpaces = " " * $padLeft
    $rightSpaces = " " * $padRight

    # Line 1: Box Top Frame
    Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor
$TrackColor -NoNewline
    Write-Host ($script:ViewModel.Glyph.TopLeft +
($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) +

```

```

$script:ViewModel.Glyph.TopRight) -ForegroundColor $BoxColor

# Line 2: Text Payload Line (Outputs ANSI natively without forced
Foreground parameters)
Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor
$TrackColor -NoNewline
Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor
$BoxColor -NoNewline
Write-Host "$leftSpaces $($script:ViewModel.Ansi.Magenta)$displayText$rightSpaces"
-NoNewline
Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $BoxColor

# Line 3: Box Bottom Frame
Write-Host "$($script:ViewModel.Glyph.Vertical) " -ForegroundColor
$TrackColor -NoNewline
Write-Host ($script:ViewModel.Glyph.BotLeft +
($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) +
$script:ViewModel.Glyph.BotRight) -ForegroundColor $BoxColor

Write-Host $script:ViewModel.Glyph.Vertical -ForegroundColor $TrackColor
}

function Write-Intro {
    param (
        [Parameter(Mandatory = $true)]
        [string]$Message,
        [switch]$Success,
        [ConsoleColor]$TrackColor = 'DarkGray'
    )
    # Terminate vertical track line
    Write-Track -Color $TrackColor -Intro

    # Extract context color
    $targetColor = if ($Success) { 'Green' } else { 'Red' }
    $color = Get-Color -Color $targetColor
    $reset = $script:ViewModel.Ansi.Reset

    # Process internal ANSI
    $processedMessage = Edit-Text -Message $Message -Color $color

    # Stream to terminal
    Write-Host "$color$processedMessage$reset"
}

function Get-SpinnerInstance {
    param (
        [char[]]$Chars,
        [string]$Message,
        [string]$Prefix,
        [string]$Color,
        [string]$Reset
    )
    # The background worker thread does not have access to parent scopes.
    # All necessary properties must be completely self-contained and
    passed as arguments.
    return [powershell]::Create().AddScript({
        param(
            [char[]]$Chrs,
            [string]$Msg,
            [string]$Pfx,
            [string]$Clr,
            [string]$Rst
        )
    })
}

```

```

        $index = 0
        while ($true) {
            $char = $Chars[$index % $Chars.Length]
            [Console]::Write("`r$Pfx $Clr$char$Rst $Msg")
            Start-Sleep -Milliseconds 100
            $index++
        }
    }).AddArgument($Chars).AddArgument($Message).AddArgument($Prefix).AddArgument($Color).AddArgument($Reset)
}

function Start-Spinner {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory = $true)]
        [string]$Message,
        [string]$Prefix,
        [string]$Color = 'BrightBlue'
    )

    if (-not $Prefix) {
        $Prefix =
"$($script:ViewModel.Glyph.BotLeft)$($script:ViewModel.Glyph.Horizontal)"
    }

    # Extract ANSI sequences from view model
    $spinnerColor = Get-Color -Color $Color
    if (-not $spinnerColor) { $spinnerColor =
$script:ViewModel.Ansi.BrightBlue }

    $spinnerFrames = [char[]]' 🌀 '

    # Build instance using our isolated creation helper function
    $instance = Get-SpinnerInstance -Chars $spinnerFrames -Message
$Message -Prefix $Prefix -Color $spinnerColor -Reset
$script:ViewModel.Ansi.Reset

    # Trigger async invocation handle loop
    [void]$instance.BeginInvoke()
    return $instance
}

function Stop-Spinner {
    param(
        [Parameter(Mandatory = $true)]
        [System.Management.Automation.PowerShell]$SpinnerInstance
    )

    # Force kill and dispose the asynchronous background pipeline thread safely
    $SpinnerInstance.Stop()
    $SpinnerInstance.Dispose()

    # Reset cursor position to column zero
    [Console]::Write("`r")

    # Extract ANSI codes from view model map
    $clearSequence = $script:ViewModel.Ansi.ClearLineToEnd

    if ($clearSequence) {
        [Console]::Write($clearSequence)
    }
    else {
        $width = if ([Console]::WindowWidth -gt 0) {
[Console]::WindowWidth - 1 } else { 80 }

```



```

        [Console]::Write((" " * $width) + "`r")
    }
}

function Get-Padding {
    param (
        [string]$Character = ' ',
        [int]$Count = 40
    )

    $Character * $Count
}

#####
# CONTROLLER: Flow Orchestration
#####
$script:evaluator = {
    param($match)
    $removedPrefixLength = $match.Groups['prefix'].Length
    $remainingPath = $match.Groups['suffix'].Value
    $terminatorType = $match.Groups['type'].Value

    if ($terminatorType -eq ':') {
        return $remainingPath
    }

    $paddingSpaces = Get-Padding -Count $removedPrefixLength
    return "${remainingPath}${paddingSpaces}"
}

function Invoke-PostProcessing {
    param(
        [Parameter(Mandatory = $true)]
        [string]$FilePath,

        [Parameter(Mandatory = $true)]
        [string]$Divider
    )

    $maxWidth = 125

    if (Test-Path $FilePath) {
        $logContent = Get-Content -Path $FilePath -Raw

        # 1. ALWAYS Scrub ANSI Escape sequences completely first
        $ansiPattern = '\x1b\[[0-9;]*[a-zA-Z]'
        $logContent = $logContent -replace $ansiPattern, ''

        # 2. Scrub the stream error wrapper string and leave an
        intentional blank line gap
        # Using `r`n ensures cross-platform line break compatibility
        on Windows systems
        $errorWrapperPattern =
        '(?m)^System\.Management\.Automation\.RemoteException\r?\n?'
        $logContent = $logContent -replace $errorWrapperPattern, "`r`n"

        # 3. Inject divider exactly BEFORE every new workspace test block starts
        $workspaceStartPattern = '(?m)^(?=.*?\bbun test Vertical\d)'
        $logContent = $logContent -replace $workspaceStartPattern, $Divider

        # 4. Strip out the monorepo workspace prefix (e.g.,
        '@pendex/color test: ')
        $workspacePrefixPattern =

```

```

'(?m)^@[a-zA-Z0-9_\-]+/[a-zA-Z0-9_\-]+ test: '
$logContent = $logContent -replace $workspacePrefixPattern, ''

# 5. Replace all literal '(fail)' strings with '(Error)'
# Parentheses must be escaped with backslashes in regex
$failPattern = '\(fail\)'
$logContent = $logContent -replace $failPattern, '(Error)'

# 6. Replace all literal '(fail)' strings with '(Error)'
# Parentheses must be escaped with backslashes in regex
$failPattern = '\(pass\)'
$logContent = $logContent -replace $failPattern, "('pass')"

# 7. Shorten path and shift remaining path by the removed prefix length
$pathPattern =
'(?m)^(?<prefix>.*?(?=pendex\\))(?<suffix>pendex\\[^\r\n]*?)(?=s*(?<type>\\|:))'
$logContent = [regex]::Replace($logContent, $pathPattern,
$script:evaluator)

# 8. Format nested test descriptions by breaking onto clean,
indented lines
$logContent = $logContent -replace ' > ', "`r`n`t> "

# 9. Search for two consecutive blank lines and reduce to one
# Three consecutive line breaks create two empty visual lines
$blankLinesPattern = '(?m)(\r?\n){3}'
$logContent = $logContent -replace $blankLinesPattern, "`r`n`r`n"

# Write the segmented content back to the log file
$logContent |
Out-File -FilePath $FilePath -Width $maxWidth -Encoding utf8 -Force
}
}

function Invoke-BunCommand {
    param(
        [Parameter(Mandatory = $true)]
        [PSCustomObject]$Config
    )
    $oldPreference = $ErrorActionPreference
    $ErrorActionPreference = 'Continue'

    try {
        # Write the initial section header directly to the log file
        $Config.LogHeader | Out-File -FilePath $Config.LogFilePath
        -Width $Config.MaxLogFilePathWidth -Encoding utf8

        # Execute main root coverage tests
        & bun test --coverage -- $Config.TestsPath 2>&1 |
        ForEach-Object { $_.ToString() } |
        Out-File -FilePath $Config.LogFilePath -Append -Width
        $Config.MaxLogFilePathWidth -Encoding utf8
        $exit1 = $LASTEXITCODE

        # Append the structural divider string to the log file
        $Config.LogDivider | Out-File -FilePath $Config.LogFilePath
        -Append -Width $Config.MaxLogFilePathWidth -Encoding utf8

        # Execute monorepo workspace tests
        & bun run --workspaces test 2>&1 |
        ForEach-Object { $_.ToString() } |
        Out-File -FilePath $Config.LogFilePath -Append -Width
        $Config.MaxLogFilePathWidth -Encoding utf8
    }
}

```

```

        $exit2 = $LASTEXITCODE

        Invoke-PostProcessing -FilePath $Config.LogFilePath -Divider
$Config.TestDivider

        if ($exit1 -ne 0) {
            return $exit1
        }
        return $exit2
    }
    catch {
        throw $_
    }
    finally {
        $ErrorActionPreference = $oldPreference
    }
}

function Invoke-TestController {
    param(
        [Parameter(Mandatory = $true)]
        [PSCustomObject]$Config
    )

    $greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"

    # Render Header
    Write-Host
    Write-Intro -Title "EXECUTING TESTS" -TrackColor $Config.TrackColor -NewLine

    # Render Step
    Write-Step -Message "Running tests and logging output to:" -Color
Cyan -TrackColor $Config.TrackColor
    Write-Box -Text $Config.LogFileName -MaxWidth
$Config.MaxLogTextWidth -TrackColor $Config.TrackColor -BoxColor
$Config.BoxColor

    # Execute Test Runner Task with Active Spinner UI
    $spinner = Start-Spinner -Message "Executing bun tests..."
    $exitCode = 1

    try {
        $exitCode = Invoke-BunCommand -Config $Config
    }
    finally {
        Stop-Spinner -SpinnerInstance $spinner
    }

    # Render Footer
    Write-Step -Message "$greenCheck Executing bun tests...
Completed." -Color Green -TrackColor $Config.TrackColor -NewLine

    $removedLogs = [ProjectModel]::CleanOldLogs()
    if ($removedLogs) {
        Write-Step -Message "$greenCheck Oldest log file(s) removed to
maintain retention cap." -Color Gray -TrackColor $Config.TrackColor
-NewLine
    }

    if ($exitCode -eq 0) {
        Write-Outro -Message "✅ Complete. All tests passed. See log

```

```

for details." -Success -TrackColor $Config.TrackColor
}
else {
    Write-Output -Message "A Complete. Some tests failed. See log
for details." -TrackColor $Config.TrackColor
}

return $exitCode
}

function Remove-OldCombinedLogs {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]
        [string]$LogPath,
        [Parameter(Mandatory = $true)]
        [PSCustomObject]$Config,
        [boolean]$IsVerbose = $false
    )

    $greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"

    # Target files matching the combined naming convention pattern
    $OldCombinedFiles = Get-ChildItem -Path $LogPath -Filter
    "**_junit_combined_*.xml" -File

    foreach ($File in $OldCombinedFiles) {
        try {
            Remove-Item -Path $File.FullName -Force -ErrorAction Stop
        }
        catch {
            Write-Error "✗Failed to remove old file $($File.FullName): $_"
        }
    }

    if ($IsVerbose) {
        Write-Step "$greenCheck Removed old combined JUnit log(s)."
        -Color $config.JUnitColor -TrackColor $config.TrackColor -NewLine
    }
}

function Resolve-LogDirectory {
    param ([string]$Path)

    if (Test-Path $Path -PathType Leaf) {
        $Path = Split-Path -Path $Path -Parent
    }

    if (-not (Test-Path $Path -PathType Container)) {
        Write-Error "✗Error. Log directory path does not exist: $Path"
        return $null
    }

    return $Path
}

function Invoke-XmlMerge {
    param (
        [System.Xml.XmlDocument]$CombinedXml,
        [System.Object[]]$XmlFiles
    )

    foreach ($file in $XmlFiles) {
        try {
            $individualXml = New-Object System.Xml.XmlDocument

```

```

        $individualXml.Load($file.FullName)

        $testSuites = $individualXml.SelectNodes("//testsuite")

        foreach ($suite in $testSuites) {
            $importedSuite = $combinedXml.ImportNode($suite, $true)
            $null = $combinedXml.DocumentElement.AppendChild($importedSuite)
        }
    }
    catch {
        Write-Error "✘Error. Failed to parse XML file:
$(file.FullName). $_"
    }
}

function Merge-JunitLogs {
    [CmdletBinding()]
    param ()

    $greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
    $IsVerbose = $PSBoundParameters.ContainsKey('Verbose') -or
($VerbosePreference -eq 'Continue')

    $config = [ProjectModel]::GetConfiguration()
    $LogPath = Resolve-LogDirectory -Path $config.LogPath
    if (-not $LogPath) { return }

    if ($IsVerbose) {
        Write-Host
        Write-Intro -Title "MERGE JUNIT FILES" -TrackColor
$config.TrackColor -NewLine
    }

    # Execute the requested old log cleanup BEFORE path processing and creation
    Remove-OldCombinedLogs -LogPath $LogPath -Config $config
-IsVerbose $IsVerbose

    # Filename timestamp settings
    $datestamp = Get-Date -Format "yyyyMMdd"
    $timestamp = Get-Date -Format "HHmmss"
    $combinedFileName = "${datestamp}_junit_combined_${timestamp}.xml"
    $combinedFilePath = Join-Path $LogPath $combinedFileName

    # Gather matching source junit test files
    $xmlFiles = @(Get-ChildItem -Path $LogPath -Filter "junit*.xml" -File |
        Where-Object { $_.Name -ne $combinedFileName -and $_.Extension
-eq '.xml' })

    if ($xmlFiles.Count -eq 0) {
        if ($IsVerbose) {
            Write-Host "✘Error. No JUnit XML files found to merge.`n"
        }
        return
    }

    # Initialize primary XML document
    $combinedXml = New-Object System.Xml.XmlDocument
    $rootNode = $combinedXml.CreateElement("testsuites")
    $null = $combinedXml.AppendChild($rootNode)

    # Process XML tree compilation via helper

```

```

Invoke-XmlMerge -CombinedXml $combinedXml -XmlFiles $xmlFiles

# File output IO routine
try {
    $combinedXml.Save($combinedFilePath)
    if ($IsVerbose) {
        Write-Step "Merging JUNIT test files to:" -Color
$config.JUnitColor -TrackColor $config.TrackColor
        Write-Box $combinedFileName -TrackColor $config.TrackColor
    }

    # Purge localized source files
    $xmlFiles | Remove-Item -Force
    if ($IsVerbose) {
        Write-Step "$greenCheck Deleted original package JUnit
Logs." -Color $config.JUnitColor -TrackColor $config.TrackColor
-NewLine
    }
}
catch {
    Write-Error "✖Error. Failed to save the combined XML file or
purge originals. $_"
}

# Final wrap-up output alerts
if ($IsVerbose) {
    Write-Outro "$greenCheck Complete." -Success -TrackColor
$config.TrackColor
}
else {
    $fallbackColor = Get-Color -Color Green
    $message = "`n$greenCheck Merging JUNIT test files complete."
    $processedMessage = Edit-Text -Message $message -Color $fallbackColor
    Write-Host $processedMessage
}
}

#####
# EXECUTION ENTRY POINT
#####
function Invoke-TerminalScriptError {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory = $true)]
        [int]$Code
    )

    $greenCheck =
"$($script:ViewModel.Ansi.Green)$($script:ViewModel.Glyph.Check)$($script:ViewModel.Ansi.Reset)"
    $IsVerbose = $PSBoundParameters.ContainsKey('Verbose') -or
($VerbosePreference -eq 'Continue')

    if ($Code -ne 0) {
        # Clear any accidental error stream pollution to keep terminal clean
        $global:Error.Clear()

        # Grab ANSI codes
        $reset = $script:ViewModel.Ansi.Reset
        $bold = $script:ViewModel.Ansi.Bold
        $red = $script:ViewModel.Ansi.Red

        # Check if the environment variable has a value
        $scriptName = $env:npm_lifecycle_event

```

```

    if (-not $scriptName) { $scriptName = "unknown" }

    if ($IsVerbose) {
        # Output custom error message to the terminal
        Write-Host "`n${red}Error${reset}: script
${bold}`"$scriptName`"${reset} exited with code $Code"
    }

}

else {
    if ($IsVerbose) {
        $color = Get-Color -Color Green
        $message = "`n$greenCheck ALL TESTING COMPLETE."
        $processedMessage = Edit-Text -Message $message -Color $color

        # Output custom error message to the terminal
        Write-Host $processedMessage
    }
}

# Exit back to the calling process without throwing built-in exceptions
[Environment]::Exit($Code)
}

```

```

function Invoke-Tests {
    # Establish UI terminal preferences
    Initialize-AnsiTerminal

    # Instantiate application configurations
    $config = [ProjectModel]::GetConfiguration()

    # Handover control to the MVVMC loop
    $exitCode = Invoke-TestController -Config $config

    # Combine junit xml files if they exist
    Merge-JunitLogs

    # Exit
    Invoke-TerminalScriptError -Code $exitCode
}

```

Invoke-Tests

```

// ██████████ < END > - ██████████
// FILE NAME: : log-tests-v01.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests-v01.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < END > - ██████████

// ██████████ < START > - ██████████
// FILE NAME: : log-tests.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < START > - ██████████
# FILE-PATH: ./utils/log-tests.ps1
#
# Requires Windows PowerShell 5.1
#
# IMPORTANT:
# Save this file explicitly as UTF-8 WITH BOM for Windows PowerShell 5.1.

```

```

# Raw UTF-8 without BOM can cause PS 5.1 to misread string literals.
#
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8

#=====
# VIEW-MODEL: State Translation & UI Definitions
#=====

# Enable ANSI/VT rendering in PS 5.1 console hosts
function Initialize-AnsiTerminal {
    [CmdletBinding()]
    param()

    # Only needed for Windows PowerShell 5.1 and earlier
    if ($PSVersionTable.PSVersion.Major -gt 5) {
        return $true
    }

    # Win32 API signatures required to toggle console VT modes
    $Signatures = @"
        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern IntPtr GetStdHandle(int nStdHandle);

        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern bool GetConsoleMode(IntPtr hConsoleHandle, out uint lpMode);

        [DllImport("kernel32.dll", SetLastError = true)]
        public static extern bool SetConsoleMode(IntPtr hConsoleHandle, uint dwMode);
"@

    # Load type only once —suppress warnings and errors
    $type = ("Win32.Win32Utils" -as [type])
    if (-not $type) {
        $type = Add-Type -MemberDefinition $Signatures `
            -Namespace "Win32" `
            -Name "Win32Utils" `
            -PassThru `
            -ErrorAction SilentlyContinue `
            2>$null
    }

    # If type failed to load, ANSI cannot be enabled
    if (-not $type) {
        return $false
    }

    # Standard output handle
    $STD_OUTPUT_HANDLE = -11
    $stdOut = $type::GetStdHandle($STD_OUTPUT_HANDLE)

    # Non-console hosts return 0 or -1 —avoid errors
    if ($stdOut -eq 0) {
        return $false
    }

    # Try to retrieve current console mode
    $mode = 0
    if (-not $type::GetConsoleMode($stdOut, [ref]$mode)) {
        # Non-interactive hosts will fail here
        return $false
    }

    # Enable VT processing + improved newline behavior

```



```

$ENABLE_VTP = 0x0004 # ENABLE_VIRTUAL_TERMINAL_PROCESSING
$newMode = $mode -bor $ENABLE_VTP

# Apply updated mode
$success = $type::SetConsoleMode($stdout, $newMode)

# Optionally ensure future consoles inherit VT support
try {
    New-Item -Path HKCU:\Console -ErrorAction SilentlyContinue | Out-Null
    Set-ItemProperty -Path HKCU:\Console -Name VirtualTerminalLevel -Type DWord -Value 1 -ErrorAction SilentlyContinue
}
catch {
    # Registry access is optional; ignore any failure
}
return $success
}

$script:ViewModel = @{
    Glyph = @{
        # Core Frames
        TopLeft      = [string][char]0x256D # ┌
        TopRight     = [string][char]0x256E # ┐
        BotLeft      = [string][char]0x2570 # └
        BotRight     = [string][char]0x256F # ┘
        Horizontal   = [string][char]0x2500 # ─
        Vertical     = [string][char]0x2502 # │
        RightTee     = [string][char]0x251C # ├
        LeftTee      = [string][char]0x2524 # ┤
        SqTopLeft    = [string][char]0x250C # ┌
        SqBotLeft    = [string][char]0x2514 # └

        # Missing Clack Connectors & State Lines
        BarMiddle    = [string][char]0x2506 # ┆ Multi-line inputs or open streams
        BarDashed    = [string][char]0x254C # ⋯ Divides sub-sections horizontally
        StepActive   = [string][char]0x25C9 # ◎ Active radial items / focused inputs

        # Indicators & Alerts
        Check        = [string][char]0x2713 # ✓ Success
        Cross        = [string][char]0x2715 # ✕ Failure / Canceled / Error state
        Info         = [string][char]0x2139 # ⓘ Instructions / Info block tooltips
        Warning      = [string][char]0x26A0 # ⚠ Warning flags

        # Pointers & Selection Geometric shapes
        Arrowhead    = [string][char]0x25B6 # ►
        PointerRight = [string][char]0x203A # ›
        Diamond      = [string][char]0x25C7 # ◇
        SolidDiamond = [string][char]0x25C6 # ◆
        Circle       = [string][char]0x25CB # ○
        SolidCircle  = [string][char]0x25CF # ●

        # Powerline Pill
        PillLeftA    = [string][char]0xE0B6 # ◀
        PillLeftB    = [string][char]0x2588 # █
        PillRightA   = [string][char]0x2588 # █
        PillRightB   = [string][char]0xE0B4 # ▶
    }
}

Ansi = @{
    # Core Modifiers & Controls
    Esc      = [char]27
    Reset    = "$([char]27)[0m"
    Bold     = "$([char]27)[1m"
    Dim      = "$([char]27)[2m"

```

```

Underline      = "$([char]27)[4m"
Invert         = "$([char]27)[7m"
Hidden         = "$([char]27)[8m"

# Standard Foreground Colors
Black          = "$([char]27)[30m"
Red            = "$([char]27)[31m"
Green          = "$([char]27)[32m"
Yellow         = "$([char]27)[33m"
Blue           = "$([char]27)[34m"
Magenta        = "$([char]27)[35m"
Cyan           = "$([char]27)[36m"
White          = "$([char]27)[37m"

# Bright Foreground Colors
BrightBlack    = "$([char]27)[90m"
Gray           = "$([char]27)[90m"
BrightRed      = "$([char]27)[91m"
BrightGreen    = "$([char]27)[92m"
BrightYellow   = "$([char]27)[93m"
BrightBlue     = "$([char]27)[94m"
BrightMagenta  = "$([char]27)[95m"
BrightCyan     = "$([char]27)[96m"
BrightWhite    = "$([char]27)[97m"

# Standard Background Colors
BgBlack        = "$([char]27)[40m"
BgRed          = "$([char]27)[41m"
BgGreen        = "$([char]27)[42m"
BgYellow       = "$([char]27)[43m"
BgBlue         = "$([char]27)[44m"
BgMagenta      = "$([char]27)[45m"
BgCyan         = "$([char]27)[46m"
BgWhite        = "$([char]27)[47m"

# Bright Background Colors
BgBrightBlack  = "$([char]27)[100m"
BgBrightRed    = "$([char]27)[101m"
BgBrightGreen  = "$([char]27)[102m"
BgBrightYellow = "$([char]27)[103m"
BgBrightBlue   = "$([char]27)[104m"
BgBrightMagenta = "$([char]27)[105m"
BgBrightCyan   = "$([char]27)[106m"
BgBrightWhite  = "$([char]27)[107m"

# Screen / Terminal Actions
ClearScreen    = "$([char]27)[2J" # Clear entire console screen window buffer
ClearLineToEnd = "$([char]27)[K" # Clear line from cursor position to right edge
CursorHome     = "$([char]27)[H"  # Reset cursor back to Top-Left position (0,0)

# Default Track Line Color
DimDarkGray    = "$([char]27)[2m$([char]27)[90m"
BgDimDarkGray  = "$([char]27)[2m$([char]27)[100m"
DarkGray       = "$([char]27)[90m"
BgDarkGray     = "$([char]27)[100m"
}

Spinner = @{
    Arc      = @('◡', '◢', '◣', '◤')
    Line     = @('|', '/', '-', '\')
    Braille  = @('⠠', '⠡', '⠢', '⠣', '⠤', '⠥', '⠦', '⠧', '⠨', '⠩', '⠪', '⠫')

# 🌀 Emoji Spinners

```

```

Moon1  = @('🌑', '🌒', '🌓', '🌔', '🌕', '🌖', '🌗', '🌘')
Moon2  = @('🌑', '🌒', '🌓', '🌔', '🌕', '🌖', '🌗', '🌘')
Clock  = @('🕒', '🕓', '🕔', '🕕', '🕖', '🕗', '🕘', '🕙', '🕚', '🕛', '🕜', '🕝', '🕞', '🕟')
Earth  = @('🌍', '🌎', '🌏')
Heart  = @('♥', '📄', '♥', '📄', '📄')

# 📄 Unicode & Terminal Symbol Spinners
Bar    = @('▬', '▮', '▯', '▰', '▱', '▲', '△', '▴', '▵', '▶', '▷')
Block  = @('█', '▒', '░', '░')
Ball    = @('(●)', '(●)', '(●)', '(●)', '(●)', '(●)')
Arcs    = @('ㄿ', 'ㄿ', 'ㄿ', 'ㄿ')
Pulse   = @('▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬', '▬')

}

Cli     = @{
    # Success / Verification
    Success = "✅" # Green Check Mark
    Check   = "✓"  # Light Check Mark

    # Errors / Failures
    Error    = "✖" # Red Cross Mark
    Cross    = "✖" # Heavy Multiplication X
    Warning  = "⚠" # Warning / Triangle

    # Information / Status
    Info     = "i"  # Information Source
    Bullet   = "•"  # Bullet Point
    Sparkle  = "❖" # Sparkles

    # Progress / Time
    Wait     = "🕒" # Hourglass Flowing Sand
    Clock    = "🕒" # Hourglass Done
    Rocket   = "🚀" # Rocket (Surrogate pair for UTF-16)

    # Cursor
    Hide     = "$([char]27)?25l"
    Show     = "$([char]27)?25h"
}

}

#####
# MODEL: Business Logic & Data Structures
#####

class ProjectPaths {
    [string]$ScriptDir
    [string]$ProjectRootDir
    [string]$TestsDir
    [string]$LogsDir

    ProjectPaths() {
        if ($PSScriptRoot) {
            $this.ScriptDir = [System.IO.Path]::GetFullPath($PSScriptRoot)
        }
        else {
            $this.ScriptDir = [System.IO.Path]::GetFullPath(".")
        }

        $this.ProjectRootDir = [ProjectPaths]::GetProjectRootPath($this.ScriptDir)
        $this.TestsDir = Join-Path $this.ProjectRootDir "tests"
        $this.LogsDir = Join-Path $this.ProjectRootDir "logs"
    }
}

```

```

static [string] GetProjectRootPath([string]$StartDir) {
    $currentDir = [System.IO.Path]::GetFullPath($StartDir)
    $firstPackageJsonFound = $null

    while ($currentDir -and (Test-Path $currentDir)) {
        if (
            (Test-Path (Join-Path $currentDir ".git")) -or
            (Test-Path (Join-Path $currentDir "package.json")) -or
            (Test-Path (Join-Path $currentDir "tsconfig.json")) -or
            (Test-Path (Join-Path $currentDir "bun.lock"))
        ) {
            return $currentDir
        }

        if ((Test-Path (Join-Path $currentDir "package.json")) -and (-not $firstPackageJsonFound)) {
            $firstPackageJsonFound = $currentDir
        }

        $parentDir = Split-Path -Path $currentDir -Parent
        if ($parentDir -eq $currentDir) {
            break
        }

        $currentDir = $parentDir
    }

    if ($firstPackageJsonFound) {
        return $firstPackageJsonFound
    }

    return [System.IO.Path]::GetFullPath($StartDir)
}
}

```

```

class TestLogConfig {
    [ProjectPaths]$Paths

    [int]$MaxLogCount
    [int]$MaxLogFileWidth
    [int]$MaxLogTextWidth
    [int]$SpinnerClearWidth

    [string]$Appendage
    [string]$LogFileName
    [string]$LogFilePath

    # Structural / theme colors
    [string]$TrackColor
    [string]$BoxColor
    [string]$SpinnerColor
    [string]$JUnitColor
    [string]$PillColor
    [string]$PillTextColor

    # Semantic / text colors
    [string]$SuccessColor
    [string]$WarningColor
    [string]$ErrorColor
    [string]$InfoColor
    [string]$MutedColor
    [string]$TextColor
    [string]$TextAccentColor

```

```

[string]$White

# Style selections
[string]$SpinnerStyle
[string]$SpinnerPolicy
[int]$SpinnerInterval

[string]$LogHeader
[string]$LogDivider
[string]$TestDivider

TestLogConfig([ProjectPaths]$paths) {
    $timestamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
    $width = 125
    $shortWidth = 50
    $dash = "-" * $width
    $star = "*" * $width
    $headerStart = "=== [1/2] PROJECT ROOT TEST START ==="
    $headerMid = "=== *[2/2]* WORKSPACE TEST START *=== "
    $workspace = "=== *[--]* WORKSPACE TEST START *=== "

    $this.Paths = $paths

    $this.MaxLogCount = 5
    $this.MaxLogFileWidth = $width
    $this.MaxLogTextWidth = $shortWidth
    $this.SpinnerClearWidth = 80

    $this.Appendage = "_tests_results.log"
    $this.LogFileName = "{0}{1}" -f $timestamp, $this.Appendage
    $this.LogFilePath = Join-Path $paths.LogsDir $this.LogFileName

    # Structural / theme colors
    $this.TrackColor = "DarkGray"
    $this.BoxColor = "Yellow"
    $this.SpinnerColor = "BrightMagenta"
    $this.JUnitColor = "Yellow"
    $this.PillColor = "Magenta"
    $this.PillTextColor = "Black"

    # Semantic / text colors
    $this.SuccessColor = "Green"
    $this.WarningColor = "Yellow"
    $this.ErrorColor = "Red"
    $this.InfoColor = "Cyan"
    $this.MutedColor = "DarkGray"
    $this.TextColor = "Gray"
    $this.TextAccentColor = "Magenta"
    $this.White = "White"

    # Style selections
    $this.SpinnerStyle = "Moon2"
    $this.SpinnerPolicy = "Logical"
    $this.SpinnerInterval = 100

    $this.LogHeader = "`r`n$dash`r`n$headerStart`r`n$dash`r`n"
    $this.LogDivider = "`r`n$dash`r`n$headerMid`r`n$dash`r`n"
    $this.TestDivider = "`r`n`r`n$star`r`n$star`r`n$workspace`r`n$star`r`n$star`r`n`r`n`r`n"
}
}

class LogManager {
    [TestLogConfig]$Config

```



```

    if (-not (Test-Path $filePath -PathType Leaf)) {
        return
    }

    $logContent = Get-Content -Path $filePath -Raw

    # 1. Strip ANSI escape sequences
    $ansiPattern = '\x1b\[[0-9; ]*[a-zA-Z]'
    $logContent = $logContent -replace $ansiPattern, ''

    # 2. Strip PowerShell stream wrapper line
    $errorWrapperPattern = '(?m)^System\.Management\.Automation\.RemoteException\r?\n?'
    $logContent = $logContent -replace $errorWrapperPattern, "`r`n"

    # 3. Inject divider before workspace blocks
    $workspaceStartPattern = '(?m)^(?=.*?\bbun test Vertical\d)'
    $logContent = $logContent -replace $workspaceStartPattern, $divider

    # 4. Strip monorepo workspace prefix
    $workspacePrefixPattern = '(?m)^@[a-zA-Z0-9_\-]+/[a-zA-Z0-9_\-]+ test: '
    $logContent = $logContent -replace $workspacePrefixPattern, ''

    # 5. Break nested test descriptions onto clean, indented lines
    $logContent = $logContent -replace ' > ', "`r`n`t> "

    # 6. Reduce 3+ line breaks to 2
    $blankLinesPattern = '(?m)(\r?\n) { 3, }'
    $logContent = $logContent -replace $blankLinesPattern, "`r`n`r`n"

    $logContent |
    Out-File -FilePath $filePath -Width $maxWidth -Encoding utf8 -Force
}
}

```

```

class BunTestRunner {
    # Public Class Properties
    [TestLogConfig]$Config
    [LogManager]$LogManager

    # Constructor
    BunTestRunner(
        [TestLogConfig]$Config,
        [LogManager]$LogManager
    ) {
        $this.Config = $Config
        $this.LogManager = $LogManager
    }

    # Public Run Method
    [int] Run() {
        $this.Config.LogHeader |
        Out-File `
            -FilePath $this.Config.LogFilePath `
            -Width $this.Config.MaxLogFileWidth `
            -Encoding utf8

        & bun test --coverage -- $this.Config.Paths.TestsDir 2>&1 |
        ForEach-Object { $_.ToString() } |
        Out-File `
            -FilePath $this.Config.LogFilePath `
            -Append `
            -Width $this.Config.MaxLogFileWidth `

```

```

        -Encoding utf8

$exit1 = $LASTEXITCODE

$this.Config.LogDivider |
Out-File `
    -FilePath $this.Config.LogFilePath `
    -Append `
    -Width $this.Config.MaxLogFileWidth `
    -Encoding utf8

& bun run --workspaces test 2>&1 |
ForEach-Object { $_.ToString() } |
Out-File `
    -FilePath $this.Config.LogFilePath `
    -Append `
    -Width $this.Config.MaxLogFileWidth `
    -Encoding utf8

$exit2 = $LASTEXITCODE

$this.LogManager.NormalizeLog()

if ($exit1 -ne 0) {
    return $exit1
}

return $exit2
}
}

#=====
# VIEW HELPERS
#=====

function Get-Color {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory = $true)]
        [string]$Color
    )

    if ($script:ViewModel.Ansi.ContainsKey($Color)) {
        return $script:ViewModel.Ansi[$Color]
    }

    return $script:ViewModel.Ansi.White
}

function Get-Padding {
    [CmdletBinding()]
    param (
        [string]$Character = ' ',
        [int]$Count = 40
    )

    $Character * $Count
}

function Edit-Text {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]

```



```

        [string]$Message,
        [string]$Color = $script:ViewModel.Ansi.White
    )

    $ansiPattern = [regex]::Escape([char]27 + "[")
    if ($Message -match $ansiPattern) {
        return $Message + $script:ViewModel.Ansi.Reset
    }

    return $Color + $Message + $script:ViewModel.Ansi.Reset
}

function Write-Track {
    [CmdletBinding(DefaultParameterSetName = 'Step')]
    param (
        [Parameter(Mandatory)]
        [TestLogConfig]$Config,

        [Parameter(ParameterSetName = 'Intro', Mandatory)]
        [switch]$Intro,

        [Parameter(ParameterSetName = 'Step', Mandatory)]
        [switch]$Step,

        [Parameter(ParameterSetName = 'Outro', Mandatory)]
        [switch]$Outro,

        [Parameter(ParameterSetName = 'Blank', Mandatory)]
        [switch]$Blank
    )

    $trackAnsi = Get-Color -Color $Config.TrackColor
    $resetAnsi = $script:ViewModel.Ansi.Reset

    if ($Blank) {
        Write-Host (
            $trackAnsi +
            $script:ViewModel.Glyph.Vertical +
            $resetAnsi
        )

        return
    }

    $glyph = $script:ViewModel.Glyph.RightTee

    if ($Intro) {
        $glyph = $script:ViewModel.Glyph.TopLeft
    }
    elseif ($Outro) {
        $glyph = $script:ViewModel.Glyph.BotLeft
    }

    Write-Host (
        $trackAnsi +
        $glyph +
        $script:ViewModel.Glyph.Horizontal +
        ' ' +
        $resetAnsi
    ) -NoNewline
}

function Write-Pill {

```

```

[CmdletBinding()]
param (
    [Parameter(Mandatory = $true)]
    [string]$Text,

    [Parameter(Mandatory = $true)]
    [TestLogConfig]$Config,

    [Parameter(ParameterSetName = 'Success')]
    [switch]$Success,

    [Parameter(ParameterSetName = 'Warning')]
    [switch]$Warning,

    [Parameter(ParameterSetName = 'Error')]
    [switch]$Err,

    [Parameter(ParameterSetName = 'Info')]
    [switch]$Info,

    [Parameter(ParameterSetName = 'Muted')]
    [switch]$Muted,

    [switch]$NoNewline
)

$pillAnsi = Get-Color -Color $Config.PillColor
$textAnsi = Get-Color -Color $Config.PillTextColor
$bgAnsi = Get-Color -Color ("Bg" + $Config.PillColor)

switch ($true) {
    $Info {
        $pillAnsi = Get-Color -Color $Config.InfoColor
        $bgAnsi = Get-Color -Color ("Bg" + $Config.InfoColor)
        break
    }
    $Success {
        $pillAnsi = Get-Color -Color $Config.SuccessColor
        $bgAnsi = Get-Color -Color ("Bg" + $Config.SuccessColor)
        break
    }
    $Warning {
        $pillAnsi = Get-Color -Color $Config.WarningColor
        $bgAnsi = Get-Color -Color ("Bg" + $Config.WarningColor)
        break
    }
    $Muted {
        $pillAnsi = Get-Color -Color $Config.MutedColor
        $textAnsi = Get-Color -Color $Config.White
        $bgAnsi = Get-Color -Color ("Bg" + $Config.MutedColor)
        break
    }
    $Err {
        $pillAnsi = Get-Color -Color $Config.ErrorColor
        $textAnsi = Get-Color -Color $Config.White
        $bgAnsi = Get-Color -Color ("Bg" + $Config.ErrorColor)
        break
    }
    default {
        $pillAnsi = Get-Color -Color $Config.PillColor
        $textAnsi = Get-Color -Color $Config.PillTextColor
        $bgAnsi = Get-Color -Color ("Bg" + $Config.PillColor)
        break
    }
}

```

```

    }
}

$resetAnsi = $script:ViewModel.Ansi.Reset

$left = "$($script:ViewModel.Glyph.PillLeftA)$($script:ViewModel.Glyph.PillLeftB)"
$right = "$($script:ViewModel.Glyph.PillRightA)$($script:ViewModel.Glyph.PillRightB)"

$output =
$pillAnsi + $left +
$bgAnsi + $textAnsi + " $Text " + $resetAnsi +
$pillAnsi + $right + $resetAnsi

if ($NoNewline) {
    Write-Host $output -NoNewline
}
else {
    Write-Host $output
}
}

```

```

function Write-Intro {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]
        [string]$Title,

        [Parameter(Mandatory = $true)]
        [TestLogConfig]$Config,

        [Parameter(ParameterSetName = 'Success')]
        [switch]$Success,

        [Parameter(ParameterSetName = 'Warning')]
        [switch]$Warning,

        [Parameter(ParameterSetName = 'Error')]
        [switch]$Err,

        [Parameter(ParameterSetName = 'Info')]
        [switch]$Info,

        [Parameter(ParameterSetName = 'Muted')]
        [switch]$Muted,

        [switch]$NewLine
    )

    Write-Track -Config $Config -Intro

    $text = "$($script:ViewModel.Ansi.Bold)$Title"

    switch ($true) {
        $Info {
            Write-Pill -Text $text -Config $Config -Info
            break
        }
        $Success {
            Write-Pill -Text $text -Config $Config -Success
            break
        }
        $Warning {
            Write-Pill -Text $text -Config $Config -Warning

```

```

        break
    }
    $Muted {
        Write-Pill -Text $text -Config $Config -Muted
        break
    }
    $Err {
        Write-Pill -Text $text -Config $Config -Err
        break
    }
    default {
        Write-Pill -Text $text -Config $Config
        break
    }
}

if ($NewLine) {
    Write-Track -Config $Config -Blank
}

}

function Write-Step {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]
        [string]$Message,

        [Parameter(Mandatory = $true)]
        [TestLogConfig]$Config,

        [Parameter(ParameterSetName = 'Success')]
        [switch]$Success,

        [Parameter(ParameterSetName = 'Warning')]
        [switch]$Warning,

        [Parameter(ParameterSetName = 'Error')]
        [switch]$Err,

        [Parameter(ParameterSetName = 'Info')]
        [switch]$Info,

        [Parameter(ParameterSetName = 'Muted')]
        [switch]$Muted,

        [switch]$NewLine,
        [switch]$NoTrack
    )

    if ($NoTrack) {
    }
    else {
        Write-Track -Config $Config -Step
    }

    $colorName = $Config.TextColor

    switch ($true) {
        $Info {
            $colorName = $Config.InfoColor
            break
        }
        $$Success {

```

```

        $colorName = $Config.SuccessColor
        break
    }
    $Warning {
        $colorName = $Config.WarningColor
        break
    }
    $Muted {
        $colorName = $Config.MutedColor
        break
    }
    default {
        $colorName = $Config.TextColor
        break
    }
}

$fallbackColor = Get-Color -Color $colorName
$processedMessage = Edit-Text -Message $Message -Color $fallbackColor

Write-Host $processedMessage
Write-Track -Config $Config -Blank
}

function Write-Box {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]
        [string]$Text,

        [Parameter(Mandatory = $true)]
        [TestLogConfig]$Config
    )

    $MaxWidth = $Config.MaxLogTextWidth

    if ($MaxWidth -lt 4) {
        $MaxWidth = 4
    }

    $trackAnsi = Get-Color -Color $Config.TrackColor
    $boxAnsi = Get-Color -Color $Config.BoxColor
    $textAnsi = Get-Color -Color $Config.TextAccentColor
    $resetAnsi = $script:ViewModel.Ansi.Reset

    $displayText = $Text
    if ($displayText.Length -gt ($MaxWidth - 4)) {
        $displayText = $displayText.Substring(0, $MaxWidth - 4)
    }

    $innerWidth = $MaxWidth - 2
    $paddingTotal = $innerWidth - $displayText.Length
    if ($paddingTotal -lt 0) {
        $paddingTotal = 0
    }

    $padLeft = [Math]::Floor($paddingTotal / 2)
    $padRight = $paddingTotal - $padLeft
    $leftSpaces = " " * $padLeft
    $rightSpaces = " " * $padRight

    $topBorder = $script:ViewModel.Glyph.TopLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) + $script:ViewModel.Glyph.TopRight
    $bottomBorder = $script:ViewModel.Glyph.BotLeft + ($script:ViewModel.Glyph.Horizontal * ($MaxWidth - 2)) + $script:ViewModel.Glyph.BotRight

```

```

Write-Host ($trackAnsi + $script:ViewModel.Glyph.Vertical + " " + $resetAnsi) -NoNewline
Write-Host ($boxAnsi + $topBorder + $resetAnsi)

Write-Host ($trackAnsi + $script:ViewModel.Glyph.Vertical + " " + $resetAnsi) -NoNewline
Write-Host ($boxAnsi + $script:ViewModel.Glyph.Vertical + $resetAnsi) -NoNewline
Write-Host ($leftSpaces + $textAnsi + $displayText + $resetAnsi + $rightSpaces) -NoNewline
Write-Host ($boxAnsi + $script:ViewModel.Glyph.Vertical + $resetAnsi)

Write-Host ($trackAnsi + $script:ViewModel.Glyph.Vertical + " " + $resetAnsi) -NoNewline
Write-Host ($boxAnsi + $bottomBorder + $resetAnsi)

Write-Host ($trackAnsi + $script:ViewModel.Glyph.Vertical + $resetAnsi)
}

```

```

function Write-Outro {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory = $true)]
        [string]$Message,

        [Parameter(Mandatory = $true)]
        [TestLogConfig]$Config,

        [Parameter(ParameterSetName = 'Success')]
        [switch]$Success,

        [Parameter(ParameterSetName = 'Warning')]
        [switch]$Warning,

        [Parameter(ParameterSetName = 'Error')]
        [switch]$Err,

        [Parameter(ParameterSetName = 'Info')]
        [switch]$Info,

        [Parameter(ParameterSetName = 'Muted')]
        [switch]$Muted
    )

    Write-Track -Config $Config -Outro

    if ($Success) {
        $color = Get-Color -Color $Config.SuccessColor
        $processedMessage = Edit-Text -Message $Message -Color $color
    }
    elseif ($Warning) {
        $color = Get-Color -Color $Config.WarningColor
        $processedMessage = Edit-Text -Message $Message -Color $color
    }
    elseif ($Err) {
        $color = Get-Color -Color $Config.ErrorColor
        $processedMessage = Edit-Text -Message $Message -Color $color
    }
    else {
        $color = Get-Color -Color $Config.ErrorColor
        $processedMessage = Edit-Text -Message $Message -Color $color
    }

    Write-Host $processedMessage
}

```

```

#=====

```

```

# SPINNER
#=====

function Test-ConsoleMeasurementAvailable {
    [CmdletBinding()]
    param()

    # ISE does not expose reliable console cursor measurements.
    if ($Host -and $Host.Name -like '*ISE*') {
        return $false
    }

    try {
        if ([Console]::IsOutputRedirected -or [Console]::IsErrorRedirected) {
            return $false
        }

        $null = [Console]::BufferWidth
        $null = [Console]::CursorLeft
        $null = [Console]::CursorTop

        return $true
    }
    catch {
        return $false
    }
}

function Get-UnicodeCodePoints {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$String
    )

    $codePoints = New-Object System.Collections.Generic.List[int]
    $characters = $String.ToCharArray()
    $index = 0

    while ($index -lt $characters.Length) {
        if (
            ($index + 1) -lt $characters.Length -and
            [char]::IsHighSurrogate($characters[$index]) -and
            [char]::IsLowSurrogate($characters[$index + 1])
        ) {
            $codePoints.Add(
                [char]::ConvertToUtf32(
                    $characters[$index],
                    $characters[$index + 1]
                )
            )

            $index += 2
        }
        else {
            $codePoints.Add([int]$characters[$index])
            $index++
        }
    }

    return $codePoints.ToArray()
}

```

```

function Test-IsVariationSelector {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    return (
        ($CodePoint -ge 0xFE00 -and $CodePoint -le 0xFE0F) -or
        ($CodePoint -ge 0xE0100 -and $CodePoint -le 0xE01EF)
    )
}

function Test-IsEmojiModifier {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    return ($CodePoint -ge 0x1F3FB -and $CodePoint -le 0x1F3FF)
}

function Test-IsRegionalIndicator {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    return ($CodePoint -ge 0x1F1E6 -and $CodePoint -le 0x1F1FF)
}

function Test-IsEmojiCore {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    return (
        ($CodePoint -ge 0x1F000 -and $CodePoint -le 0x1FAFF) -or
        ($CodePoint -ge 0x2600 -and $CodePoint -le 0x27BF)
    )
}

function Test-IsCombiningMark {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    if ($CodePoint -le 0xFFFF) {
        try {
            $category = [System.Globalization.CharUnicodeInfo]::GetUnicodeCategory(
                [string][char]$CodePoint,
                0
            )

            if (
                $category -eq [System.Globalization.UnicodeCategory]::NonSpacingMark -or
                $category -eq [System.Globalization.UnicodeCategory]::SpacingCombiningMark -or

```



```

        $category -eq [System.Globalization.UnicodeCategory]::EnclosingMark
    ) {
        return $true
    }
}
catch {
    # Continue to explicit combining-mark range checks.
}
}

return (
    ($CodePoint -ge 0x1AB0 -and $CodePoint -le 0x1AFF) -or
    ($CodePoint -ge 0x1DC0 -and $CodePoint -le 0x1DFF) -or
    ($CodePoint -ge 0x20D0 -and $CodePoint -le 0x20FF) -or
    ($CodePoint -ge 0xFE20 -and $CodePoint -le 0xFE2F)
)
}

function Test-IsClusterExtension {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int]$CodePoint
    )

    return (
        (Test-IsVariationSelector -CodePoint $CodePoint) -or
        (Test-IsEmojiModifier -CodePoint $CodePoint) -or
        (Test-IsCombiningMark -CodePoint $CodePoint) -or
        $CodePoint -eq 0x20E3 -or
        ($CodePoint -ge 0xE0020 -and $CodePoint -le 0xE007F)
    )
}

function Get-LogicalClusterWidth {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [int[]]$CodePoints
    )

    if ($CodePoints.Count -eq 0) {
        return 0
    }

    $hasZwj = $false
    $hasKeycap = $false
    $hasEmoji = $false
    $regionalIndicatorCount = 0
    $baseCodePoint = $null
    $hasVariationSelector = $false

    foreach ($codePoint in $CodePoints) {
        if ($codePoint -eq 0x200D) {
            $hasZwj = $true
            continue
        }

        if (Test-IsVariationSelector -CodePoint $codePoint) {
            $hasVariationSelector = $true
            continue
        }
    }
}

```

```

    if ($codePoint -eq 0x20E3) {
        $hasKeycap = $true
        continue
    }

    if (Test-IsClusterExtension -CodePoint $codePoint) {
        continue
    }

    if ($null -eq $baseCodePoint) {
        $baseCodePoint = $codePoint
    }

    if (Test-IsRegionalIndicator -CodePoint $codePoint) {
        $regionalIndicatorCount++
    }

    if (Test-IsEmojiCore -CodePoint $codePoint) {
        $hasEmoji = $true
    }
}

if ($null -eq $baseCodePoint) {
    return 0
}

# Flag sequences: two regional indicators form one wide flag glyph.
if ($regionalIndicatorCount -ge 2) {
    return 2
}

# A single regional indicator is still treated as emoji-width.
if ($regionalIndicatorCount -eq 1) {
    return 2
}

# Keycap sequences, for example: 1⌨ , #⌨ , *⌨ .
if ($hasKeycap) {
    return 2
}

# Emoji joined by U+200D are treated as one wide glyph.
if ($hasZwj) {
    return 2
}

# Logical rendering contract:
# ♡, ♥, and ❤ occupy one column.
if ($baseCodePoint -eq 0x2764) {
    return 1
}

# Some BMP symbols become emoji only when paired with VS16.
if (
    $hasVariationSelector -and
    (
        $baseCodePoint -eq 0x00A9 -or
        $baseCodePoint -eq 0x00AE -or
        $baseCodePoint -eq 0x203C -or
        $baseCodePoint -eq 0x2049 -or
        $baseCodePoint -eq 0x2122 -or
        $baseCodePoint -eq 0x2139 -or
        ($baseCodePoint -ge 0x0023 -and $baseCodePoint -le 0x0039)
    )

```

```

    )
} {
    return 2
}

if ($hasEmoji) {
    return 2
}

# East Asian Wide / Fullwidth ranges.
if (
    ($baseCodePoint -ge 0x1100 -and $baseCodePoint -le 0x115F) -or
    ($baseCodePoint -ge 0x2329 -and $baseCodePoint -le 0x232A) -or
    ($baseCodePoint -ge 0x2E80 -and $baseCodePoint -le 0xA4CF) -or
    ($baseCodePoint -ge 0xAC00 -and $baseCodePoint -le 0xD7A3) -or
    ($baseCodePoint -ge 0xF900 -and $baseCodePoint -le 0FAFF) -or
    ($baseCodePoint -ge 0xFE10 -and $baseCodePoint -le 0xFE19) -or
    ($baseCodePoint -ge 0xFE30 -and $baseCodePoint -le 0xFE6F) -or
    ($baseCodePoint -ge 0xFF00 -and $baseCodePoint -le 0xFF60) -or
    ($baseCodePoint -ge 0xFFE0 -and $baseCodePoint -le 0xFFE6) -or
    ($baseCodePoint -ge 0x20000 -and $baseCodePoint -le 0x3FFFD)
) {
    return 2
}

return 1
}

function Get-LogicalStringWidth {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$String
    )

    $codePoints = @(Get-UnicodeCodePoints -String $String)
    $totalWidth = 0
    $index = 0

    while ($index -lt $codePoints.Count) {
        $cluster = New-Object System.Collections.Generic.List[int]
        $current = $codePoints[$index]

        # A standalone extension has no independent display width.
        if (
            (Test-IsClusterExtension -CodePoint $current) -or
            $current -eq 0x200D
        ) {
            $index++
            continue
        }

        # Pair regional indicators into one flag cluster where possible.
        if (
            (Test-IsRegionalIndicator -CodePoint $current) -and
            (($index + 1) -lt $codePoints.Count) -and
            (Test-IsRegionalIndicator -CodePoint $codePoints[$index + 1])
        ) {
            $cluster.Add($current)
            $cluster.Add($codePoints[$index + 1])
            $index += 2
        }
        else {

```



```

        if (($bufferWidth - $left) -lt $minimumRemainingColumns) {
            return $null
        }

        [Console]::Write($String)

        $afterLeft = [Console]::CursorLeft
        $afterTop = [Console]::CursorTop

        $delta = (($afterTop - $top) * $bufferWidth) + ($afterLeft - $left)

        # A negative value indicates that the host scrolled or otherwise
        # changed cursor state unexpectedly. Reject the measurement.
        if ($delta -lt 0) {
            [Console]::SetCursorPosition($left, $top)
            return $null
        }

        # Erase the probe and restore the original cursor position.
        [Console]::SetCursorPosition($left, $top)

        if ($delta -gt 0) {
            [Console]::Write(' ' * $delta)
            [Console]::SetCursorPosition($left, $top)
        }

        return $delta
    }
    catch {
        return $null
    }
}

function Get-DisplayStringWidth {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$String,

        [Parameter(Mandatory)]
        [TestLogConfig]$Config
    )

    $logicalWidth = Get-LogicalStringWidth -String $String
    $policy = $Config.SpinnerPolicy

    if ([string]::IsNullOrEmpty($policy)) {
        $policy = 'Logical'
    }

    if ($policy -eq 'Logical') {
        return $logicalWidth
    }

    if ($policy -eq 'PreferLogical') {
        return $logicalWidth
    }

    $physicalWidth = Get-RenderedStringWidth -String $String

    if ($null -eq $physicalWidth -or $physicalWidth -le 0) {
        return $logicalWidth
    }
}

```

```

        if ($policy -eq 'Physical') {
            return $physicalWidth
        }

        if ($policy -eq 'PreferPhysical') {
            return $physicalWidth
        }

        # LogicalOnDisagreement
        if ($physicalWidth -eq $logicalWidth) {
            return $physicalWidth
        }

        return $logicalWidth
    }

function Format-SpinnerFrame {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Frame,

        [Parameter(Mandatory)]
        [ValidateRange(0, 1000)]
        [int]$TargetWidth,

        [Parameter(Mandatory)]
        [ValidateRange(0, 1000)]
        [int]$DisplayWidth
    )

    if ($DisplayWidth -ge $TargetWidth) {
        return $Frame
    }

    return $Frame + (' ' * ($TargetWidth - $DisplayWidth))
}

function Format-SpinnerFrames {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [ValidateNotNullOrEmpty()]
        [string[]]$Frames,

        [Parameter(Mandatory)]
        [TestLogConfig]$Config
    )

    $widths = New-Object System.Collections.Generic.List[int]
    $maximumWidth = 0

    foreach ($frame in $Frames) {
        if ($null -eq $frame) {
            throw 'Spinner frames cannot contain null values.'
        }

        $width = Get-DisplayStringWidth `
            -String $frame `
            -Config $Config

        $widths.Add([int]$width)
    }

```

```

        if ($width -gt $maximumWidth) {
            $maximumWidth = $width
        }
    }

    $formattedFrames = New-Object System.Collections.Generic.List[string]

    for ($index = 0; $index -lt $Frames.Length; $index++) {
        $formattedFrames.Add(
            (Format-SpinnerFrame `
                -Frame $Frames[$index] `
                -TargetWidth $maximumWidth `
                -DisplayWidth $widths[$index])
        )
    }

    return $formattedFrames.ToArray()
}

function New-SpinnerState {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Message,

        [Parameter(Mandatory)]
        [TestLogConfig]$Config,

        [string]$Prefix
    )

    if ([string]::IsNullOrEmpty($Prefix)) {
        $rawPrefix = (
            "$($script:ViewModel.Glyph.BotLeft)" +
            "$($script:ViewModel.Glyph.Horizontal)"
        )
    }
    else {
        $rawPrefix = $Prefix
    }

    $style = $Config.SpinnerStyle

    if ([string]::IsNullOrEmpty($style)) {
        $style = 'Moon2'
    }

    if (-not $script:ViewModel.Spinner.ContainsKey($style)) {
        $availableStyles = (
            $script:ViewModel.Spinner.Keys |
            Sort-Object
        ) -join ', '

        throw "Unknown spinner style '$style'. Available styles: $availableStyles"
    }

    $rawFrames = @(
        $script:ViewModel.Spinner[$style] |
        ForEach-Object {
            [string]$_
        }
    )
}

```

```

if ($rawFrames.Count -eq 0) {
    throw "Spinner style '$style' does not contain any frames."
}

$trackAnsi = Get-Color -Color $Config.TrackColor

if (-not $trackAnsi) {
    $trackAnsi = $script:ViewModel.Ansi.Gray
}

$spinnerAnsi = Get-Color -Color $Config.SpinnerColor

if (-not $spinnerAnsi) {
    $spinnerAnsi = $script:ViewModel.Ansi.BrightMagenta
}

$interval = $Config.SpinnerInterval

if ($interval -lt 25) {
    $interval = 100
}

$useAnsi = $false

try {
    if (Initialize-AnsiTerminal) {
        $useAnsi = $true
    }
}
catch {
    $useAnsi = $false
}

return [PSCustomObject]@{
    Frames          = Format-SpinnerFrames `
        -Frames $rawFrames `
        -Config $Config

    Message         = $Message
    Prefix          = $trackAnsi + $rawPrefix + $script:ViewModel.Ansi.Reset
    Color           = $spinnerAnsi
    Reset           = $script:ViewModel.Ansi.Reset
    ClearLineToEnd  = $script:ViewModel.Ansi.ClearLineToEnd
    UseAnsi         = [bool]$useAnsi
    IntervalMilliseconds = [int]$interval
}
}

function Get-SpinnerInstance {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [PSCustomObject]$State
    )

    return [System.Management.Automation.PowerShell]::Create().AddScript({
        param(
            [PSCustomObject]$SpinnerState
        )

        if ($SpinnerState.UseAnsi) {
            [Console]::Write("${char}[27][?251")

```



```

    }

    try {
        $index = 0

        while ($true) {
            $frame = $SpinnerState.Frames[
                $index % $SpinnerState.Frames.Length
            ]

            [Console]::Write(
                "`r${$SpinnerState.Prefix} " +
                "${$SpinnerState.Color}$frame" +
                "${$SpinnerState.Reset} " +
                "${$SpinnerState.Message}" +
                "${$SpinnerState.ClearLineToEnd}"
            )

            Start-Sleep -Milliseconds $SpinnerState.IntervalMilliseconds
            $index++
        }
    }
    finally {
        if ($SpinnerState.UseAnsi) {
            [Console]::Write("${[char]27}[?25h")
        }

        [Console]::Write("`r")

        if ($SpinnerState.ClearLineToEnd) {
            [Console]::Write($SpinnerState.ClearLineToEnd)
        }
    }
}

}).AddArgument($State)
}

function Start-Spinner {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Message,

        [Parameter(Mandatory)]
        [TestLogConfig]$Config,

        [string]$Prefix
    )

    $spinnerState = New-SpinnerState `
        -Message $Message `
        -Config $Config `
        -Prefix $Prefix

    $instance = Get-SpinnerInstance -State $spinnerState

    [void]$instance.BeginInvoke()

    return $instance
}

function Stop-Spinner {
    [CmdletBinding()]
    param(

```

```

        [AllowNull()]]
        [System.Management.Automation.PowerShell]$SpinnerInstance,

        [Parameter(Mandatory)]
        [TestLogConfig]$Config
    )

    if ($null -ne $SpinnerInstance) {
        try {
            $SpinnerInstance.Stop()
        }
        catch {
            # The worker may already be stopped.
        }

        try {
            $SpinnerInstance.Dispose()
        }
        catch {
            # Ignore disposal failures during cleanup.
        }
    }

    try {
        [Console]::Write("`r")

        $clearSequence = [string]$script:ViewModel.Ansi.ClearLineToEnd

        if ($clearSequence) {
            [Console]::Write($clearSequence)
            return
        }

        $width = $Config.SpinnerClearWidth

        if ($width -lt 1) {
            $width = 80
        }

        [Console]::Write((' ' * $width) + "`r")
    }
    catch {
        # No usable console is available.
    }
}

#####
# CONTROLLER
#####

function Invoke-TestController {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [TestLogConfig]$Config
    )

    $logManager = New-Object LogManager($Config)
    $runner = New-Object BunTestRunner($Config, $logManager)

    $exitCode = 1
    $successColor = "$(Get-Color -Color $config.SuccessColor)"
    $check = "(${script:ViewModel.Glyph.Check})"

```

```

$greenCheck = "$successColor$check$($script:ViewModel.Ansi.Reset)"

# Model setup
$logManager.EnsureLogDirectory()
$logManager.RemoveOldCombinedLogs()

$STRINGS = @{
    Title = "EXECUTING TESTS"
    Step1 = @{
        Start = "Running tests and logging output to:"
        FileName = "$($config.LogFileName)"
    }
    Step2 = @{
        Spinner = "Executing bun tests..."
        Complete = "$greenCheck Executing bun tests... Completed."
    }
    Step3 = "$greenCheck Oldest log file(s) removed to maintain retention cap."
    Outro = @{
        Success = "✅ Complete. All tests passed. See log for details."
        Failure = "⚠ Complete. Some tests failed. See log for details."
    }
}

# View: Header
Write-Host
Write-Intro -Title $STRINGS.Title -Config $config -NewLine -Err

# View: Step 1: Log target info
Write-Step -Message $STRINGS.Step1.Start -Config $config -Info
Write-Box -Text $STRINGS.Step1.FileName -Config $config

# View + Controller: Step 2: Spinner-wrapped execution
$spinner = Start-Spinner -Message $STRINGS.Step2.Spinner -Config $config

try {
    # Step 2.A.: Perform work here.
    $exitCode = $runner.Run()
}
finally {
    # Step 2.B.: Stop Spinner here.
    Stop-Spinner -SpinnerInstance $spinner -Config $Config
}

# View: Step 2 (Cont): Completion step
Write-Step -Message $STRINGS.Step2.Complete -Config $config -Success -NewLine

# Model + View: Step 3: retention cleanup
$removedLogs = $logManager.CleanOldLogs()

if ($removedLogs) {
    Write-Step -Message $STRINGS.Step3 -Config $config -Muted -NewLine
}

# View: Step 4: Final outro
if ($exitCode -eq 0) {
    Write-Outro -Message $STRINGS.Outro.Success -Config $config -Success
}
else {
    Write-Outro -Message $STRINGS.Outro.Failure -Config $config -Warning
}

return $exitCode
}

```

```

#####
# EXECUTE
#####

function Start-TestLog {
    [CmdletBinding()]
    param()

    # Assign default exit code
    $script:finalExitCode = 1

    $ansiEnabled = $false
    $paths = $null
    $config = $null

    try {
        $ansiEnabled = Initialize-AnsiTerminal

        $paths = New-Object ProjectPaths
        $config = New-Object TestLogConfig($paths)

        # Cursor ownership belongs to the application lifecycle, not an
        # individual spinner. This prevents the cursor from reappearing
        # between multiple spinner instances.
        if ($ansiEnabled) {
            [Console]::Write("${[char]27}[?251")
        }

        $script:finalExitCode = Invoke-TestController -Config $config
    }
    catch {
        $originalError = $_

        try {
            # Initialize only what is required to render a controlled failure
            # message if startup failed before Config construction completed.
            if ($null -eq $config) {
                if ($null -eq $paths) {
                    $paths = New-Object ProjectPaths
                }

                $config = New-Object TestLogConfig($paths)
            }

            $message1 = (
                "$($script:ViewModel.Cli.Warning) " +
                'Unhandled error while executing log-tests.ps1'
            )

            $message2 = (
                "$($script:ViewModel.Cli.Error) " +
                $originalError.Exception.Message
            )

            Write-Track -Config $config -Blank
            Write-Intro -Title $message1 -Config $config -Warning -NewLine
            Write-Outro -Message $message2 -Config $config -Err
        }
        catch {
            # Do not replace the original failure with an error-rendering
            # failure. Emit the original exception to the PowerShell stream.
            Write-Error $originalError
        }
    }
}

```

```

    }

    $script:finalExitCode = 1
}
finally {
    # Show the cursor
    if ($ansiEnabled) {
        try {
            [Console]::Write("${[char]27}[?25h")
        }
        catch {
            # No usable console is available.
        }
    }
}

exit $script:finalExitCode
}

```

Start-TestLog

```

// ██████████ < END > ██████████
// FILE NAME: : log-tests.ps1
// TYPE      : .PS1
// PATH      : utils/log-tests.ps1
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < END > ██████████

// ██████████ < START > ██████████
// FILE NAME: : log-tests.fish
// TYPE      : .FISH
// PATH      : utils/log-tests.fish
// PROCESSED: 2026-09-06T21:55:32.591Z
// ██████████ < START > ██████████

#!/usr/bin/env fish

# FILE-PATH: ./log-tests.fish

# Make sure this .fish script file is explicitly saved with the
# encoding UTF-8 *without* BOM. Fish is UTF-8 built-in and a BOM
# would be parsed as part of the first command.

# Ensure console handles UTF-8 box characters correctly
if not string match -qir 'utf-?8' -- "$LANG$LC_ALL$LC_CTYPE"
    set -gx LC_ALL C.UTF-8
end

#-----
# Model
#-----

function Get-ScriptConfiguration
    set -l scriptFolder (status dirname)

    if test -z "$scriptFolder"
        set scriptFolder "."
    end

    set scriptFolder (path resolve $scriptFolder)
    set -l timestamp (date +"%Y-%m-%d_%H%M%S")
    set -l appendage "_tests.results.log"
    set -l logFileName "$timestamp$appendage"

```

```

set -g config_Sleep      1
set -g config_MaxLogs    5
set -g config_MaxWidth   50
set -g config_LogMaxWidth 100

set -g config_ScriptFolder "$scriptFolder"
set -g config_TestsPath    "$scriptFolder/tests"
set -g config_LogDir       "$scriptFolder/logs"
set -g config_Appendage    "$appendage"
set -g config_LogFile      "$logFileName"
set -g config_LogFilePath  "$scriptFolder/logs/$logFileName"

set -g config_BorderColor "Gray"
set -g config_BoxColor    "Yellow"
set -g config_BarKey      "heavy_block"
end

function Initialize-Environment --argument-names LogDir
  if not test -d "$LogDir"
    mkdir -p "$LogDir"
  end
end

function Manage-Log --argument-names LogDir MaxLogs Appendage
  set -l existingLogs (find "$LogDir" -maxdepth 1 -type f -name
    "$Appendage" | sort)

  set -l currentCount (count $existingLogs)

  if test $currentCount -gt $MaxLogs
    set -l logsToRemoveCount (math $currentCount - $MaxLogs)

    # Select the oldest ones and force remove them
    rm -f -- $existingLogs[1..$logsToRemoveCount]
    return 0
  end
  return 1
end

function Invoke-Bun-Test --description 'Run every test suite (root +
every workspace package) via bun run test'
  # Forward any extra args ($argv, e.g. --coverage) straight through.
  # Deliberately does NOT scope this to a single tests/ directory —
  # `bun run test` (package.json) is the one place that knows how to
  # run root AND every packages/*/tests suite with a correctly
  # combined exit code. Reimplementing that scoping here (as an
  # earlier version of this script did, with `bun test $TestsPath`)
  # is exactly what silently limited every run to the root suite —
  # the fix is for this script to stop knowing the shape of "run
  # everything" at all, not to widen its own copy of that knowledge.
  bun run test $argv
end

function Get-Test-Results --argument-names LogFilePath LogMaxWidth BarKey
  Invoke-Bun-Test &| Format-Log $LogMaxWidth $BarKey >$LogFilePath

  # Capture the exit code cleanly
  return $pipestatus[1]
end

#-----
# View-Model - Inspired by @clack/prompts
#-----

```



```

end
echo
end

#-----
# Controller - Main Script Orchestration
#-----

function Invoke-Tests
    # Model
    Get-ScriptConfiguration
    Initialize-Environment $config_LogDir

    # View-Model
    set -l prefix (Get-Terminalline 'BotLeft')(Get-Terminalline 'Horizontal')

    # View
    Write-Header
    Start-Spinner 'Executing bun tests...' "$prefix"

    # Controller
    Get-Test-Results $config_TestsPath $config_LogFilePath
$config_LogMaxWidth $config_BarKey
    set -l exitCode $status

    # View
    sleep $config_Sleep
    Stop-Spinner
    Write-Summary $exitCode

    exit 0
end

#-----
# Execute
#-----

Invoke-Tests

// _____< END >-_____
// FILE NAME: : log-tests.fish
// TYPE      : .FISH
// PATH      : utils/log-tests.fish
// PROCESSED: 2026-09-06T21:55:32.591Z
// _____< END >-_____

```